

# Security Audit and Hardening Guide

Project: HavenWorld | Repository: [github.com/dboone323/HavenWorld](https://github.com/dboone323/HavenWorld) | License: GPL-3.0

Author: Daniel | Date: September 18, 2026 | Status: **Pre-Alpha Security Review**

Stack: Node.js 22 LTS | Express 5 | Socket.io 4 | Prisma | PostgreSQL 16 | Redis 7 |  
Nginx | Oracle Cloud ARM

---

## Security Critical: Read Before Proceeding

This guide contains production-grade security configurations. Apply all  **HIGH** risk items before any public alpha launch. Never expose a development build with default secrets, open ports, or disabled authentication to the public internet. Every code snippet and configuration block in this guide is specific to the HavenWorld tech stack and must be tested in a staging environment before applying to production.

## Table of Contents

### 1. Security Architecture Overview

1.1 Defense-in-Depth Model | 1.2 Trust Model | 1.3 Security Posture Summary | 1.4 Scope

### 2. Server Infrastructure Hardening **LINUX**

2.1 SSH Hardening | 2.2 Automatic Security Updates | 2.3 fail2ban | 2.4 Oracle Cloud Firewall & UFW

### 3. Nginx Security Configuration **LINUX**

3.1 Security Headers | 3.2 Rate Limiting | 3.3 Request Size Limits | 3.4 TLS Configuration

## 4. Express Application Security [MAC](#) [LINUX](#)

4.1 Security Middleware | 4.2 Input Validation | 4.3 XSS Prevention | 4.4 CSRF Prevention  
| 4.5 Error Handling

## 5. WebSocket Security (Socket.io) [MAC](#) [LINUX](#)

5.1 CSWSH Prevention | 5.2 Auth at Handshake | 5.3 Event Validation | 5.4 Rate Limiting  
| 5.5 Idle Management | 5.6 Room Authorization

## 6. JWT Security [MAC](#)

6.1 Token Configuration | 6.2 Refresh Token Rotation | 6.3 Storage Security | 6.4 Logout  
& Session Termination

## 7. Game Logic Security [MAC](#) [LINUX](#)

7.1 Server-Authoritative Movement | 7.2 Economy Exploits | 7.3 P2P Trade | 7.4 Fishing  
Anti-Cheat | 7.5 Invite Codes | 7.6 Admin Privileges

## 8. Database Security [LINUX](#)

8.1 PostgreSQL Hardening | 8.2 Redis Hardening | 8.3 Backup Security

## 9. Dependency Security [MAC](#)

9.1 npm audit | 9.2 Dependabot | 9.3 GPL-3.0 Compliance

## 10. Monitoring & Incident Response [LINUX](#)

10.1 Sentry Security Events | 10.2 Suspicious Activity Detection | 10.3 Security Logging

## 11. Security Testing Tools [MAC](#) [LINUX](#)

11.1 OWASP ZAP | 11.2 npm audit & Snyk | 11.3 wscat | 11.4 Custom Pen Test Scripts

## 12. Pre-Alpha Security Checklist

30+ item verification table | Security Debt Tracking

---

# 1. Security Architecture Overview

## 1.1 Defense-in-Depth Model

HavenWorld employs a five-layer defense model. No single layer is treated as sufficient on its own. Each layer independently validates, filters, and limits what it

receives from the layer above it. If one layer is bypassed or misconfigured, the others continue to enforce security boundaries. The attack surface is reduced at every hop from the public internet to the PostgreSQL datastore.

| Layer                          | Component                | Responsibilities                                                                                            | Threats Mitigated                                                      |
|--------------------------------|--------------------------|-------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| <b>Layer 1 — Edge</b>          | Cloudflare Pages / Proxy | DDoS mitigation, CDN, HTTPS termination for frontend, bot protection                                        | Volumetric DDoS, scraping, malicious bot traffic                       |
| <b>Layer 2 — Reverse Proxy</b> | Nginx                    | TLS termination, security headers, rate limiting, WebSocket upgrade, request size enforcement               | Protocol-level attacks, clickjacking, MIME sniffing, slow-read attacks |
| <b>Layer 3 — Application</b>   | Express 5 + Socket.io 4  | Authentication, authorization, input validation, CORS, CSRF, session management, business logic enforcement | Injection, XSS, CSRF, CSWSH, auth bypass, privilege escalation         |
| <b>Layer 4 — ORM</b>           | Prisma ORM               | Parameterized queries, type-safe database access, transaction atomicity                                     | SQL injection, race conditions, partial-update exploits                |
| <b>Layer 5 — Datastore</b>     | PostgreSQL 16 + Redis 7  | Localhost-only binding, dedicated low-privilege DB user, query timeouts, encrypted backups                  | Unauthorized direct access, runaway queries, data exfiltration         |

## 1.2 Trust Model

HavenWorld is built on the principle of **zero client trust**. The game server is the single source of truth for all game state. The following rules are non-negotiable and enforced throughout this guide:

- **All client data is untrusted.** Every value from the client — position, item IDs, coin amounts, room IDs — is validated server-side before being acted upon or stored.

- **The server is authoritative.** Player positions, inventory states, economy balances, and room membership are computed and stored server-side. The client receives the result, not the other way around.
- **JWT payloads are not trusted for sensitive fields.** The admin role, account status, and ban state are always re-read from PostgreSQL on sensitive operations, never taken from the JWT claim.
- **Socket data set by the client is never trusted.** Fields on `socket.data` are only set server-side after successful validation. Socket events are treated as unauthenticated input until the middleware chain confirms otherwise.
- **Nginx is trusted; the client is not.** The `X-Forwarded-For` IP is only used after `app.set('trust proxy', 1)` is configured, meaning Nginx is the only trusted forwarder.

### 1.3 Security Posture Summary Table

| What Is Protected           | How It Is Protected                                                     | Risk If Not Done                            | Risk Level |
|-----------------------------|-------------------------------------------------------------------------|---------------------------------------------|------------|
| Authentication credentials  | bcrypt hashing, httpOnly cookies, JWT algorithm pinning                 | Account takeover, credential theft          | ● HIGH     |
| Player economy (coins/gems) | Prisma \$transaction atomicity, server-side balance checks, weekly caps | Duplication exploits, game economy collapse | ● HIGH     |
| WebSocket sessions          | Origin validation, JWT at handshake, per-event authorization            | CSWSH attack, unauthorized game actions     | ● HIGH     |
| Admin privileges            | Role from DB only, separate middleware, all actions logged              | Privilege escalation, platform takeover     | ● HIGH     |
| Player movement             | Server-authoritative validation, navmesh checks, speed cap              | Speed hacks, wall clipping, unfair gameplay | ● HIGH     |
| Chat messages               | Zod validation, DOMPurify sanitization, rate                            | XSS injection, spam, harassment             | ● MEDIUM   |

| What Is Protected          | How It Is Protected                                                        | Risk If Not Done                            | Risk Level      |
|----------------------------|----------------------------------------------------------------------------|---------------------------------------------|-----------------|
|                            | limiting                                                                   |                                             |                 |
| SSH access                 | ED25519 keys, no root login, fail2ban, non-standard port                   | Server compromise, data loss                | ● <b>HIGH</b>   |
| Database                   | Localhost binding, least-privilege user, query timeouts, encrypted backups | Data exfiltration, ransom, corruption       | ● <b>HIGH</b>   |
| TLS/HTTPS                  | TLS 1.2/1.3 only, strong ciphers, HSTS preload, OCSP stapling              | MITM attacks, credential interception       | ● <b>HIGH</b>   |
| Dependencies               | npm audit in CI, Dependabot, license compliance check                      | Supply chain attack, known CVE exploitation | ● <b>MEDIUM</b> |
| Fishing / minigame rewards | Server-generated results, timing validation, reward caps                   | Automated reward farming, economy inflation | ● <b>MEDIUM</b> |
| Invite codes               | Cryptographic randomness, single-use atomicity, 30-day expiry              | Unauthorized registrations, invite farming  | ● <b>LOW</b>    |

## 1.4 Scope of This Guide

This guide covers the full application and infrastructure stack that you directly control. The following are **in scope**: Oracle Cloud VM hardening, Nginx configuration, Express/Socket.io application security, JWT management, game logic validation, PostgreSQL and Redis hardening, dependency auditing, and security monitoring.

The following are **out of scope** for this guide (managed externally): Cloudflare DDoS mitigation and WAF rules (configured in the Cloudflare dashboard, covered in Part 2), Oracle Cloud networking-level firewall rules managed via the OCI console, GitHub repository access controls (team permissions, branch protection — covered in Part 1), and email provider (Resend) security settings.

---

## 2. Server Infrastructure Hardening LINUX

### 2.1 SSH Hardening

SSH is the primary administrative entry point to your Oracle Cloud VM. A compromised SSH configuration is a complete server compromise. Apply all of the following before any other configuration step.

#### Step 1 — Generate an ED25519 key pair on your local machine:

```
# Run on your LOCAL machine (Mac or Linux), not the server ssh-keygen -t
ed25519 -C "havenworld-deploy" -f ~/.ssh/havenworld_ed25519 # Copy the
public key to the server (replace IP with your Oracle VM IP) ssh-copy-id -i
~/.ssh/havenworld_ed25519.pub ubuntu@YOUR_VM_IP # Test login with the new
key BEFORE changing sshd_config ssh -i ~/.ssh/havenworld_ed25519
ubuntu@YOUR_VM_IP
```

#### Step 2 — Harden /etc/ssh/sshd\_config on the server:

```
# /etc/ssh/sshd_config – HavenWorld hardened configuration # IMPORTANT: Keep
your current session open while editing. # Test with a SECOND SSH session
before closing the first. Port 2222 AddressFamily inet # Authentication
PasswordAuthentication no PubkeyAuthentication yes AuthorizedKeysFile
.ssh/authorized_keys PermitRootLogin no PermitEmptyPasswords no
ChallengeResponseAuthentication no UsePAM yes # Session limits MaxAuthTries
3 MaxSessions 5 LoginGraceTime 30 ClientAliveInterval 300 ClientAliveCountMax
2 # Restrict to known users only AllowUsers ubuntu # Disable unused
features X11Forwarding no AllowTcpForwarding no GatewayPorts no
PermitUserEnvironment no # Logging SyslogFacility AUTH LogLevel VERBOSE
```

#### Step 3 — Restart SSH and verify:

```
# Validate config syntax before restarting sudo sshd -t # Restart SSH
service sudo systemctl restart sshd # Verify SSH is listening on port 2222
sudo ss -tlnp | grep 2222 # From a NEW terminal window (keep old session
open), test login: ssh -i ~/.ssh/havenworld_ed25519 -p 2222 ubuntu@YOUR_VM_IP
# Only close the original session after successful test login
```



#### **Security Critical: Never Close Your Only SSH Session**

Always have a second SSH session open and verified before restarting sshd. If you lock yourself out of an Oracle Cloud VM, you must use the OCI console's "Cloud Shell" or instance recovery console to regain access, which may require a reboot and temporary password reset.

#### Step 4 — Update your local SSH config for convenience:

```
# ~/.ssh/config on your LOCAL machine Host havenworld      HostName YOUR_VM_IP
User ubuntu      Port 2222      IdentityFile ~/.ssh/havenworld_ed25519
ServerAliveInterval 60 # Now connect with just: ssh havenworld
```

## 2.2 Automatic Security Updates LINUX

Oracle Cloud VMs run Ubuntu. Enable unattended security updates to automatically patch kernel vulnerabilities, OpenSSL, and system libraries without manual intervention.

```
# Install unattended-upgrades sudo apt update && sudo apt install -y
unattended-upgrades apt-listchanges # Enable the service sudo dpkg-
reconfigure --priority=low unattended-upgrades
```

**Configure /etc/apt/apt.conf.d/50unattended-upgrades:**

```
// /etc/apt/apt.conf.d/50unattended-upgrades Unattended-Upgrade::Allowed-
Origins {      "${distro_id}:${distro_codename}-security";      "$
${distro_id}ESMApps:${distro_codename}-apps-security";      "${distro_id}ESM:$
${distro_codename}-infra-security"; }; // Do NOT auto-upgrade non-security
packages // "${distro_id}:${distro_codename}-updates"; Unattended-
Upgrade::Package-Blacklist {      // Add specific packages to exclude if
needed }; // Automatically reboot at 3:00 AM if needed for kernel updates
Unattended-Upgrade::Automatic-Reboot "true"; Unattended-Upgrade::Automatic-
Reboot-Time "03:00"; // Remove unused kernel packages Unattended-
Upgrade::Remove-Unused-Kernel-Packages "true"; Unattended-Upgrade::Remove-
New-Unused-Dependencies "true"; // Email notification (optional – set if you
have a system MTA) // Unattended-Upgrade::Mail "daniel@yourdomain.com"; //
Verbose logging Unattended-Upgrade::Verbose "false"; Unattended-
Upgrade::Debug "false";
```

**Configure /etc/apt/apt.conf.d/20auto-upgrades:**

```
APT::Periodic::Update-Package-Lists "1"; APT::Periodic::Download-Upgradeable-
Packages "1"; APT::Periodic::AutocleanInterval "7";
APT::Periodic::Unattended-Upgrade "1";
```

**Test the configuration:**

```
# Dry-run to see what would be upgraded sudo unattended-upgrades --dry-run --
debug # Check upgrade log after first run sudo cat /var/log/unattended-
upgrades/unattended-upgrades.log
```

## 2.3 fail2ban Configuration LINUX

fail2ban monitors log files for repeated failed authentication attempts and automatically bans offending IP addresses using iptables or nftables. Install and configure it to protect SSH, Nginx login endpoints, and HavenWorld's custom auth API.

```
# Install fail2ban sudo apt install -y fail2ban # Check service status sudo
systemctl enable fail2ban sudo systemctl start fail2ban
```

**Create /etc/fail2ban/jail.local (overrides defaults safely):**

```
# /etc/fail2ban/jail.local – HavenWorld specific configuration [DEFAULT] #
Ban IPs for 1 hour on detection bantime = 3600 # Look back over the last 10
```

```
minutes for failures findtime = 600 # Allow 5 retries before banning maxretry
= 5 # Use iptables-multiport for blocking banaction = iptables-multiport #
Email alert (optional) # destemail = daniel@yourdomain.com # action = %
(action_mwl)s [sshd] enabled = true port = 2222 filter = sshd logpath
= /var/log/auth.log maxretry = 5 bantime = 3600 findtime = 600 [nginx-http-
auth] enabled = true port = http,https filter = nginx-http-auth
logpath = /var/log/nginx/error.log maxretry = 5 [nginx-limit-req] enabled
= true port = http,https filter = nginx-limit-req logpath =
/var/log/nginx/error.log maxretry = 10 bantime = 7200 findtime = 600
[havenworld-auth] enabled = true port = http,https filter =
havenworld-auth logpath = /var/log/nginx/access.log maxretry = 10 findtime =
600 bantime = 3600
```

### Create custom HavenWorld filter `/etc/fail2ban/filter.d/havenworld-`

#### `auth.conf:`

```
# /etc/fail2ban/filter.d/havenworld-auth.conf # Bans IPs that POST to
/api/auth/login and receive a 401 or 429 response # 10+ attempts within
findtime triggers a ban [Definition] failregex = ^<HOST> .* "POST
/api/auth/login HTTP/\d\.\d" (401|429) .*$ ignoreregex =
```

### Verify and manage bans:

```
# Check status of all jails sudo fail2ban-client status # Check HavenWorld-
specific jail sudo fail2ban-client status havenworld-auth # Manually unban
an IP sudo fail2ban-client set havenworld-auth unbanip 1.2.3.4 # Test ban
trigger (from a separate machine or using curl loop) for i in {1..12}; do
curl -s -o /dev/null -w "%{http_code}\n" \ -X POST
https://YOUR_DOMAIN/api/auth/login \ -H "Content-Type:
application/json" \ -d '{"username":"test","password":"wrongpass"}' done
# After iteration 10, subsequent requests should return 403 (banned by
iptables)
```

## 2.4 Oracle Cloud Firewall (Security Groups) & UFW LINUX

Oracle Cloud applies security rules at the network infrastructure level (Security Lists and Network Security Groups). These are your outermost firewall, operating before traffic reaches your VM. Add UFW (Uncomplicated Firewall) as a second enforcement layer directly on the OS.

### Oracle Cloud Security List — required ingress rules only:

| Direction | Protocol | Port | Source                  | Purpose                       |
|-----------|----------|------|-------------------------|-------------------------------|
| Ingress   | TCP      | 2222 | Your IP only (CIDR /32) | SSH (hardened port)           |
| Ingress   | TCP      | 80   | 0.0.0.0/0               | HTTP (redirect to HTTPS only) |
| Ingress   | TCP      | 443  | 0.0.0.0/0               | HTTPS + WSS                   |
| Egress    | All      | All  | 0.0.0.0/0               | Allow all outbound            |



## Verify via OCI CLI:

```
# Get your compartment OCID from OCI console, then: oci network security-list
get \ --security-list-id ocid1.securitylist.oc1... \ --query 'data.
{"ingress":"ingress-security-rules","egress":"egress-security-rules"}
```

## Configure UFW as the OS-level second layer:

```
# Install UFW if not present sudo apt install -y ufw # Set defaults: deny
all inbound, allow all outbound sudo ufw default deny incoming sudo ufw
default allow outgoing # Allow only required ports sudo ufw allow 2222/tcp
comment "SSH hardened port" sudo ufw allow 80/tcp comment "HTTP (redirect
only)" sudo ufw allow 443/tcp comment "HTTPS and WSS" # Enable UFW (you will
be prompted to confirm) sudo ufw enable # Verify rules sudo ufw status
verbose
```

### Important: Oracle Cloud iptables Override

Oracle Cloud Ubuntu images include pre-configured iptables rules in /etc/iptables/rules.v4. Run `sudo iptables -L INPUT -n --line-numbers` to verify no rules are blocking ports 80 and 443. If UFW conflicts with Oracle's default rules, you may need to flush and re-apply: `sudo iptables -F` followed by `sudo ufw reload`.

---

## 3. Nginx Security Configuration LINUX

### 3.1 Security Headers

Nginx is responsible for injecting all HTTP security headers. These headers instruct browsers to enforce strict security policies before any JavaScript or application code executes. The following is the complete /etc/nginx/sites-available/havenworld configuration:

```
# /etc/nginx/nginx.conf - global settings user www-data; worker_processes
auto; pid /run/nginx.pid; events {      worker_connections 1024; } http {
include      /etc/nginx/mime.types;      default_type  application/octet-
stream;      # — Security: Remove version disclosure
server_tokens off;      # — Rate Limit Zones
(defined globally, used in server blocks) —      limit_req_zone
$binary_remote_addr zone=login:10m rate=5r/m;      limit_req_zone
$binary_remote_addr zone=api:10m      rate=100r/m;      limit_req_zone
$binary_remote_addr zone=ws:10m      rate=10r/m;      # — Logging
log_format main
'$remote_addr - $remote_user [$time_local] '      '"$request"
```

```

$status $body_bytes_sent ' "$http_referer"
"$http_user_agent"; access_log /var/log/nginx/access.log main;
error_log /var/log/nginx/error.log warn; include /etc/nginx/sites-
enabled/*; } # /etc/nginx/sites-available/havenworld # — HTTP: Redirect to
HTTPS only server { listen 80;
server_name YOUR_IP.nip.io; location /.well-known/acme-challenge/ {
root /var/www/certbot; } location / { return 301
https://$host$request_uri; } } # — HTTPS: Main server block
server { listen 443 ssl;
http2 on; server_name YOUR_IP.nip.io; # — TLS Certificates (Let's
Encrypt via Certbot) ssl_certificate
/etc/letsencrypt/live/YOUR_IP.nip.io/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/YOUR_IP.nip.io/privkey.pem;
ssl_trusted_certificate /etc/letsencrypt/live/YOUR_IP.nip.io/chain.pem;
# — TLS Protocol & Cipher Configuration
ssl_protocols TLSv1.2 TLSv1.3; ssl_ciphers ECDHE-ECDSA-AES128-GCM-
SHA256:ECDSA-AES128-GCM-SHA256:ECDSA-AES256-GCM-SHA384:ECDSA-AES-
AES256-GCM-SHA384:ECDSA-CHACHA20-POLY1305:ECDSA-RSA-CHACHA20-
POLY1305:DHE-RSA-AES128-GCM-SHA256; ssl_prefer_server_ciphers off;
ssl_session_timeout 1d; ssl_session_cache shared:SSL:10m;
ssl_session_tickets off; # — OCSP Stapling
ssl_stapling on;
ssl_stapling_verify on; resolver 1.1.1.1 1.0.0.1 valid=300s;
resolver_timeout 5s; # — Security Headers
add_header Strict-Transport-
Security "max-age=63072000; includeSubDomains; preload" always;
add_header X-Frame-Options "DENY" always; add_header X-Content-Type-
Options "nosniff" always; add_header Referrer-Policy "strict-origin-when-
cross-origin" always; add_header Content-Security-Policy "default-src
'self'; script-src 'self'; connect-src 'self' wss://YOUR_IP.nip.io; img-src
'self' data:; style-src 'self' 'unsafe-inline'; font-src 'self'; frame-
ancestors 'none'; form-action 'self'; base-uri 'self';" always;
add_header Permissions-Policy "camera=(), microphone=(), geolocation=(),
payment=(), usb=()" always; add_header X-Permitted-Cross-Domain-Policies
"none" always; # — Request Size Limits
client_max_body_size 1m;
client_body_timeout 12s; client_header_timeout 12s; # — HTTP Method
Restriction # (Enforced per location
block – see below) # — API Proxy with Rate Limiting
location /api/auth/ { limit_req
zone=login burst=5 nodelay; limit_req_status 429; add_header
Retry-After 60 always; # Only allow relevant HTTP methods
limit_except POST GET OPTIONS { deny all; }
proxy_pass http://127.0.0.1:3000; proxy_http_version 1.1;
proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme; } location /api/ {
limit_req zone=api burst=20 nodelay; limit_req_status 429;
proxy_pass http://127.0.0.1:3000; proxy_http_version 1.1;
proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme; } # — WebSocket
(Socket.io) location /socket.io/ {
limit_req zone=ws burst=5 nodelay; limit_req_status 429;
proxy_pass http://127.0.0.1:3000; proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection
"upgrade"; proxy_set_header Host $host; proxy_set_header X-
Real-IP $remote_addr; proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for; proxy_set_header X-Forwarded-Proto
$scheme; # Must be greater than Socket.io pingInterval (10s)

```

```
# Set to 1 hour to allow persistent WebSocket connections
proxy_read_timeout    3600s;        proxy_send_timeout    3600s;
proxy_connect_timeout  75s;        }    # — Health Check (no logging)
                                location /api/health
{    access_log off;        proxy_pass http://127.0.0.1:3000;    } }
```

### Verify and reload Nginx:

```
# Test configuration syntax sudo nginx -t # Reload without dropping
connections sudo systemctl reload nginx # Verify TLS configuration curl -I
https://YOUR_IP.nip.io # Look for: Strict-Transport-Security, X-Frame-
Options, X-Content-Type-Options
```

## 3.2 Rate Limiting

Rate limiting zones are defined globally in `nginx.conf` (shown above) and applied per-location. The three zones protect distinct attack surfaces:

| Zone Name | Applies To   | Rate               | Burst | Threat Mitigated               |
|-----------|--------------|--------------------|-------|--------------------------------|
| login     | /api/auth/*  | 5 req/min per IP   | 5     | Brute-force credential attacks |
| api       | /api/*       | 100 req/min per IP | 20    | API scraping, automated abuse  |
| ws        | /socket.io/* | 10 req/min per IP  | 5     | WebSocket connection flood     |

## 3.3 Request Size Limits

Large request body attacks can exhaust server memory and CPU. HavenWorld has no legitimate use case for large uploads at this stage. Enforce strict size limits at the Nginx layer before data reaches Express:

- `client_max_body_size 1m` — Nginx-level hard limit on all request bodies. Nginx returns 413 before passing the request to Node.js.
- Express additionally enforces `express.json({ limit: '50kb' })` at the application layer (see Section 4.1).
- Socket.io enforces `maxHttpBufferSize: 64 * 1024` (64KB) for WebSocket frames (see Section 5.4).

- `client_body_timeout 12s` — disconnect slow-write (slow-loris) attack clients that trickle data.

## 3.4 TLS Configuration

TLS is already configured via Certbot from Part 2. This section audits the TLS configuration to ensure it achieves an SSL Labs "A+" grade. All cipher configuration is included in the Nginx block above in Section 3.1.

```
# Test auto-renewal (dry-run, no certificate changes) sudo certbot renew --dry-run # Verify renewal cron job exists sudo crontab -l | grep certbot # If not present, add: sudo crontab -e # Add: 0 2 * * * /usr/bin/certbot renew --quiet --post-hook "systemctl reload nginx" # Test TLS grade via curl (checks for TLS 1.0/1.1 downgrade vulnerability) curl -svo /dev/null --tlsv1.0 https://YOUR_IP.nip.io 2>&1 | grep "SSL connection" # Expected: connection failed (TLS 1.0 disabled) curl -svo /dev/null --tlsv1.2 https://YOUR_IP.nip.io 2>&1 | grep "SSL connection" # Expected: SSL connection using TLSv1.2 / ...
```

### Tip: SSL Labs Testing

Run a free SSL Labs scan at [ssllabs.com/ssltest/](https://ssllabs.com/ssltest/) targeting your `.nip.io` domain. With the cipher configuration in Section 3.1, you should receive an "A+" grade. The key requirements for A+ are: HSTS with preload and includeSubDomains, TLS 1.3 support, and no deprecated protocols. If you score below A, compare your cipher list against the Mozilla SSL Configuration Generator (modern profile).

---

## 4. Express Application Security MAC LINUX

### 4.1 Security Middleware Stack

The Express application security middleware stack must be ordered correctly. Helmet and CORS must run first, before any route handlers. Rate limiters must run before business logic.

```
# Install required security packages npm install helmet express-rate-limit cors # Install input validation npm install zod # Install XSS sanitization npm install xss // src/server/middleware/security.ts import express, { type Application } from 'express'; import helmet from 'helmet'; import cors from 'cors'; import rateLimit from 'express-rate-limit'; const ALLOWED_ORIGINS = [
```

```

'https://havenworld.pages.dev', // Development only – gated by NODE_ENV
check below ]; if (process.env.NODE_ENV !== 'production')
{   ALLOWED_ORIGINS.push('http://localhost:5173'); } // — CORS Policy
export const
corsOptions: cors.CorsOptions = {   origin: (origin, callback) => { //
Allow requests with no origin (server-to-server, curl in dev) // In
production, always require an origin   if (!origin &&
process.env.NODE_ENV !== 'production') {   return callback(null, true);
}   if (origin && ALLOWED_ORIGINS.includes(origin)) {   return
callback(null, true);   }   callback(new Error(`CORS_ORIGIN_BLOCKED: $
{origin}`)); },   credentials: true, // Required for httpOnly cookie
delivery   methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
allowedHeaders: ['Content-Type', 'Authorization', 'X-CSRF-Token'],
exposedHeaders: ['X-Request-Id'],   maxAge: 86400, // Cache
preflight for 24 hours }; // — General API Rate Limiter
export const apiRateLimiter
= rateLimit({   windowMs: 15 * 60 * 1000, // 15 minutes   max: 100,
standardHeaders: true,   legacyHeaders: false,   message: { error:
'TOO_MANY_REQUESTS', retryAfter: 900 },   skip: (req) => req.path ===
'/api/health', // Never rate-limit health check }); // — Strict Auth Rate
Limiter
export const
authRateLimiter = rateLimit({   windowMs: 15 * 60 * 1000, // 15 minutes
max: 10,   standardHeaders: true,   legacyHeaders: false,   message: { error:
'AUTH_RATE_LIMIT_EXCEEDED', retryAfter: 900 }, }); // — Apply All Security
Middleware
export function
applySecurityMiddleware(app: Application): void { // 1. Trust Nginx as the
single trusted proxy for correct IP extraction   app.set('trust proxy', 1);
// 2. Helmet – sets all standard security headers //   (X-Frame-Options,
X-Content-Type-Options, X-DNS-Prefetch-Control, //   referrer-policy,
permissions-policy, x-powered-by removal, etc.)
app.use(helmet({   contentSecurityPolicy: false, // We set CSP in Nginx
(avoids duplication)   hsts: false, // HSTS set in Nginx
with preload flag   crossOriginEmbedderPolicy: true, })); // 3. CORS –
must come before route handlers   app.use(cors(corsOptions)); // 4. Body
parser with strict size limit   app.use(express.json({ limit: '50kb' }));
app.use(express.urlencoded({ extended: true, limit: '50kb' })); // 5. Rate
limiters   app.use('/api/', apiRateLimiter);   app.use('/api/auth/',
authRateLimiter); }
// src/server/index.ts – Application entry point
import express from
'express'; import { createServer } from 'http'; import
{ applySecurityMiddleware } from './middleware/security.js'; import
{ initializeSocketio } from './socket/index.js'; // — Startup Security
Assertions
if (!
process.env.JWT_SECRET) {   throw new Error('FATAL: JWT_SECRET environment
variable is not set. Refusing to start. '); } if (!process.env.DATABASE_URL) {
throw new Error('FATAL: DATABASE_URL environment variable is not set.
Refusing to start. '); } if (!process.env.REDIS_URL) {   throw new
Error('FATAL: REDIS_URL environment variable is not set. Refusing to
start. '); } if (process.env.NODE_ENV === 'production' &&
process.env.JWT_SECRET.length < 64) {   throw new Error('FATAL: JWT_SECRET
must be at least 64 hex characters (32 bytes) in production. '); } const app
= express(); const httpServer = createServer(app);
applySecurityMiddleware(app); initializeSocketio(httpServer); // ... route
registration

```

## 4.2 Input Validation Audit

Every API endpoint and Socket.io event handler must validate its input with Zod before processing. Raw client data is never passed to Prisma, business logic functions, or broadcast to other clients without validation.

```
// src/server/validation/schemas.ts import { z } from 'zod'; import xss from 'xss'; // — Reusable field validators

const UUIDField = z.string().uuid('Must be a valid UUID'); const UUIDArray = z.array(UUIDField).min(1).max(20); const SanitizedString = (max: number) => z.string().min(1).max(max).transform((val) => xss(val.trim(), { whitelist: {}, stripIgnoreTag: true })); // — Auth Schemas

export const RegisterSchema = z.object({ username: z.string().min(3, 'Username must be at least 3 characters').max(20, 'Username cannot exceed 20 characters').regex(/^([a-zA-Z0-9_]+)$/, 'Username may only contain letters, numbers, and underscores'), password: z.string().min(8, 'Password must be at least 8 characters').max(128, 'Password cannot exceed 128 characters'), inviteCode: z.string().min(6).max(16).regex(/^([A-Za-z0-9_-]+)$/, 'Invalid invite code format') }); export const LoginSchema = z.object({ username: z.string().min(3).max(20), password: z.string().min(1).max(128), }); // — Chat Schema

export const ChatMessageSchema = z.object({ message: SanitizedString(200), roomId: UUIDField, }); // — Movement Schema

export const MoveSchema = z.object({ x: z.number().min(-500).max(500), y: z.number().min(0).max(100), z: z.number().min(-500).max(500), roomId: UUIDField, }); // — Trade Schema

export const TradeSchema = z.object({ targetUserId: UUIDField, offeredItemIds: UUIDArray, requestedItemIds: UUIDArray, }); // — Room Join Schema

export const JoinRoomSchema = z.object({ roomId: UUIDField, }); // — Fishing Schema

export const FishingResultSchema = z.object({ tensionValue: z.number().min(0).max(1), castTime: z.number().int().min(0).max(30000), // ms, max 30 seconds });

// src/server/middleware/validateBody.ts import { type Request, type Response, type NextFunction } from 'express'; import { type ZodSchema, ZodError } from 'zod'; // Generic body validator middleware factory export function validateBody(schema: ZodSchema) { return (req: Request, res: Response, next: NextFunction): void => { const result = schema.safeParse(req.body); if (!result.success) { const formatted = (result.error as ZodError).errors.map((e) => ({ field: e.path.join('.'), message: e.message, })); res.status(400).json({ error: 'VALIDATION_FAILED', fields: formatted, }); return; } // Replace req.body with the validated (and sanitized) data req.body = result.data; next(); }; }

// Usage in route: // router.post('/register', validateBody(RegisterSchema), registerHandler);
```



### Why Prisma Prevents SQL Injection

Prisma ORM uses parameterized queries exclusively. It never interpolates user input directly into SQL strings. Even if Zod validation were bypassed, a call like `prisma.user.findUnique({ where: { username: userInput } })` passes `userInput` as a bound parameter — the database treats it as data, never as executable SQL. However, Zod validation is still required because it prevents business logic abuse (negative numbers, excessively long strings, malformed UUIDs) that Prisma cannot protect against.

## 4.3 XSS Prevention

Cross-site scripting (XSS) occurs when user-supplied content is rendered as HTML in another user's browser. HavenWorld has multiple content channels (chat, guestbook, usernames) that must each be hardened.

- **Chat messages:** Sanitize with the `xss` npm package server-side before storing in the database AND before broadcasting to other clients. The `Zod SanitizedString` transformer (shown above) handles this automatically.
- **Guestbook entries:** Strip all HTML tags — allow plain text only. Use the same `SanitizedString` transformer with `whitelist {}` (empty, meaning zero allowed tags).
- **Usernames:** Enforced alphanumeric + underscore only at registration via `RegisterSchema`. This regex `/^[a-zA-Z0-9_]+$/` structurally prevents HTML injection in usernames.
- **Frontend rendering:** In `Babylon.js` UI components, always set text via `textContent` or `Babylon's TextBlock.text` property, never via `innerHTML` with user-provided values.
- **Defense-in-depth:** Even if sanitization is bypassed, the Content-Security-Policy header (set in Nginx) blocks execution of injected scripts by disallowing inline scripts and restricting script sources to `'self'`.

## 4.4 CSRF Prevention

Cross-Site Request Forgery (CSRF) tricks a logged-in user's browser into making authenticated requests to your server from a malicious website. HavenWorld uses three complementary defenses:

```
// src/server/middleware/csrf.ts import { type Request, type Response, type
NextFunction } from 'express'; import crypto from 'crypto'; // Double-Submit
Cookie Pattern for CSRF protection // 1. On login: generate a CSRF token, set
it as a NON-httpOnly cookie // so JavaScript can read it and include it in
request headers. // 2. On state-changing requests: Express reads the token
from both // the cookie AND the X-CSRF-Token header and compares them. //
A cross-site attacker cannot read the cookie (SameSite=Strict blocks //
cookie delivery), so they cannot forge the header match. export function
generateCsrfToken(): string { return
crypto.randomBytes(32).toString('hex'); } export function setCsrCookie(res:
Response, token: string): void { res.cookie('csrf_token', token,
{ httpOnly: false, // Must be readable by JavaScript (intentional)
secure: true, sameSite: 'strict', maxAge: 7 * 24 * 60 * 60 *
1000, // 7 days (matches refresh token) }); } export function
csrfProtection(req: Request, res: Response, next: NextFunction): void { //
Skip CSRF check for GET, HEAD, OPTIONS (safe methods) if (['GET', 'HEAD',
'OPTIONS'].includes(req.method)) { return next(); } const
cookieToken = req.cookies?.csrf_token; const headerToken = req.headers['x-
csrf-token']; if (!cookieToken || !headerToken)
{ res.status(403).json({ error: 'CSRF_TOKEN_MISSING' }); return; }
// Use timingSafeEqual to prevent timing attacks on token comparison const
cookieBuf = Buffer.from(cookieToken, 'utf8'); const headerBuf =
Buffer.from(headerToken as string, 'utf8'); if (cookieBuf.length !==
headerBuf.length || !crypto.timingSafeEqual(cookieBuf, headerBuf)) {
res.status(403).json({ error: 'CSRF_TOKEN_INVALID' }); return; }
next(); }
```

The refresh token cookie itself uses SameSite=Strict, which prevents the browser from including it in cross-site requests — providing an independent CSRF layer even if the double-submit pattern were disabled:

```
// Refresh token cookie – set in /api/auth/login and /api/auth/refresh
res.cookie('refresh_token', refreshToken, { httpOnly: true, // Not
accessible to JavaScript secure: true, // HTTPS only sameSite:
'strict', // Prevents CSRF – cookie not sent on cross-site requests
maxAge: 7 * 24 * 60 * 60 * 1000, path: '/api/auth', // Cookie only sent
to auth endpoints });
```

## 4.5 Error Handling & Information Leakage

Error messages in production must never reveal implementation details. Stack traces, file paths, Express version strings, and database column names are all valuable reconnaissance data for an attacker. Implement a centralized error handler as the last middleware registered:

```
// src/server/middleware/errorHandler.ts import { type Request, type
Response, type NextFunction } from 'express'; // Centralized error handler –
must be registered LAST in Express // (four-argument middleware is Express's
```



```

error handler signature) export function globalErrorHandler( err: unknown,
req: Request, res: Response, _next: NextFunction, ): void { // Log full
error internally (with stack trace) for debugging
console.error({ timestamp: new Date().toISOString(), method:
req.method, path: req.path, userId: (req as any).user?.id ??
'unauthenticated', error: err instanceof Error ? err.message :
String(err), // Stack trace only in non-production environments ...
(process.env.NODE_ENV !== 'production' && { stack: err instanceof Error
? err.stack : undefined, })), }); // Never send stack traces or
internal messages to clients const statusCode = (err as any)?.status ??
(err as any)?.statusCode ?? 500; res.status(statusCode).json({ error:
statusCode === 500 ? 'INTERNAL_SERVER_ERROR' : (err as any)?.message ??
'ERROR', // No stack, no file paths, no library version information });
} // Custom 404 handler – register before globalErrorHandler export function
notFoundHandler(req: Request, res: Response): void { // Return JSON, not
Express's default HTML (which reveals "Express" in the body)
res.status(404).json({ error: 'NOT_FOUND', path: req.path }); }
// src/server/routes/auth.ts – Login timing hardening import bcrypt from
'bcrypt'; import { prisma } from '../prisma.js'; // ❌ VULNERABLE: Short-
circuits when user not found, reveals timing difference // if (user === null)
return res.status(401).json({ error: 'User not found' }); // await
bcrypt.compare(password, user.passwordHash); // ✅ CORRECT: Always perform
bcrypt.compare regardless of whether user exists // This ensures login takes
the same amount of time for invalid users vs wrong passwords const DUMMY_HASH
= '$2b$12$dummyhashfortimingnormalizationXXXXXXXXXXXXXXXXXXXX'; export
async function loginHandler(req, res) { const { username, password } =
req.body; // Already validated by Zod middleware const user = await
prisma.user.findUnique({ where: { username } }); // Always hash, even if
user not found – prevents user enumeration via timing const hashToCompare =
user?.passwordHash ?? DUMMY_HASH; const passwordMatch = await
bcrypt.compare(password, hashToCompare); if (!user || !passwordMatch) {
// Generic message – does NOT reveal whether username or password was wrong
return res.status(401).json({ error:
'AUTHENTICATION_FAILED' }); } // ... issue tokens }

```

## 5. WebSocket Security (Socket.io) MAC LINUX

### 5.1 CSWSH Prevention (Cross-Site WebSocket Hijacking) 🚫 HIGH

#### ⚠️ Security Critical: CSWSH Attack Explanation

A Cross-Site WebSocket Hijacking (CSWSH) attack works as follows: a malicious website at <https://evil.com> includes JavaScript that opens a WebSocket connection to [wss://YOUR\\_IP.nip.io](wss://YOUR_IP.nip.io). The browser automatically includes the victim's cookies (including the refresh token) in the WebSocket upgrade HTTP request. If the server does not validate the Origin header, it accepts this

connection as authenticated with the victim's identity, allowing the attacker to send any game event (steal items, drain coins, send chats) on the victim's behalf. This is different from regular CSRF because WebSockets are not protected by SameSite cookies alone.

```
// src/server/socket/index.ts – Socket.io server initialization import
{ Server } from 'socket.io'; import type { Server as HttpServer } from
'http'; const ALLOWED_ORIGINS = ['https://havenworld.pages.dev']; if
(process.env.NODE_ENV !== 'production') {
ALLOWED_ORIGINS.push('http://localhost:5173'); } export function
initializeSocketio(httpServer: HttpServer): Server { const io = new
Server(httpServer, { cors: { origin: ALLOWED_ORIGINS,
credentials: true, }, // — Security: Size Limits
maxHttpBufferSize: 64 * 1024, //
64KB max WebSocket frame size // — Security: Disable compression
(prevents CRIME/BREACH) — perMessageDeflate: false, // —
Heartbeat Configuration — pingTimeout:
20000, // 20 seconds pingInterval: 10000, //
10 seconds // — Transport Policy
transports: ['websocket'],
// Force WebSocket only (no polling fallback)
// Polling fallback weakens Origin protection }); // — CSWSH Origin
Validation Middleware io.use((socket,
next) => { const origin = socket.handshake.headers.origin; if (!
origin || !ALLOWED_ORIGINS.includes(origin))
{ console.warn({ event: 'ORIGIN_FORBIDDEN', origin:
origin ?? 'none', ip: socket.handshake.address, timestamp:
new Date().toISOString(), }); return next(new
Error('ORIGIN_FORBIDDEN')); } next(); }); // — Authentication
Middleware (runs after Origin check) io.use(authSocketMiddleware); // — Idle Timeout Middleware

io.use(idleTimeoutMiddleware); io.on('connection', (socket) =>
{ registerSocketHandlers(socket, io); }); return io; }
```

## 5.2 Authentication Enforcement at Handshake

```
// src/server/socket/authMiddleware.ts import jwt from 'jsonwebtoken'; import
type { Socket } from 'socket.io'; interface JWTPayload { userId: string;
username: string; iat: number; exp: number; } export async function
authSocketMiddleware( socket: Socket, next: (err?: Error) => void, ):
Promise<void> { const token = socket.handshake.auth?.token as string |
undefined; if (!token) { socket.emit('connect_error',
'AUTH_REQUIRED'); return next(new Error('AUTH_REQUIRED')); } try {
// Algorithm pinning prevents algorithm confusion attacks (e.g., none, RS256
with HS256 secret) const payload = jwt.verify(token,
process.env.JWT_SECRET!, { algorithms: ['HS256'], }) as JWTPayload;
// Attach user data server-side – never trust socket.data set from client
socket.data.user = { id: payload.userId, username:
payload.username, }; socket.data.currentRoomId = null; // Set
server-side on JOIN_ROOM socket.data.violations = 0; // Anti-
cheat violation counter next(); } catch (err) { // Distinguish
between expired and invalid tokens for client UX const message = (err as
```

```
Error).name === 'TokenExpiredError' ? 'TOKEN_EXPIRED' :
'AUTH_INVALID'; socket.emit('connect_error', message); next(new
Error(message)); } }
```

## 5.3 Event Validation & Authorization

Every Socket.io event handler must pass through three gates: payload validation (Zod), room authorization, and permission check. The following table documents each event in HavenWorld:

| Event           | Payload Fields Validated                              | Authorization Check                                            | Risk            |
|-----------------|-------------------------------------------------------|----------------------------------------------------------------|-----------------|
| JOIN_ROOM       | roomId (UUID)                                         | Room exists; if private loft, check DB access mode             | ● <b>HIGH</b>   |
| PLAYER_MOVE     | x, y, z (bounded numbers), roomId (UUID)              | socket.data.currentRoomId === payload.roomId; speed validation | ● <b>HIGH</b>   |
| SEND_CHAT       | message (sanitized string $\leq 200$ ), roomId (UUID) | socket.data.currentRoomId === payload.roomId; user not muted   | ● <b>MEDIUM</b> |
| START_FISHING   | roomId (UUID), spotId (UUID)                          | User is in room; spot exists and is active in that room        | ● <b>MEDIUM</b> |
| FISHING_RESULT  | tensionValue (0–1), castTime (int $\leq 30000$ ms)    | Session ID matches server-stored fishing session for this user | ● <b>MEDIUM</b> |
| TRADE_INITIATE  | targetUserId (UUID), offeredItemIds, requestedItemIds | Both users in same room; target user exists                    | ● <b>HIGH</b>   |
| TRADE_CONFIRM   | tradeId (UUID)                                        | Trade exists, belongs to this user, in LOCKED state            | ● <b>HIGH</b>   |
| RING_DOORBELL   | loftId (UUID)                                         | Loft exists; caller not banned from that loft                  | ● <b>LOW</b>    |
| ADMIN_KICK_USER | targetUserId (UUID), reason (string $\leq 200$ )      | socket.data.user.role === 'admin' (re-read from DB)            | ● <b>HIGH</b>   |

```
// src/server/socket/handlers/moveHandler.ts import type { Socket, Server }
from 'socket.io'; import { MoveSchema } from '../../validation/schemas.js';
const MAX_SPEED = 5; // World units per second const
SPEED_HACK_MULTIPLIER = 2; export function registerMoveHandler(socket:
```

```

Socket, io: Server): void {    let lastPosition = { x: 0, y: 0, z: 0 };    let
lastMoveTime = Date.now();    socket.on('PLAYER_MOVE', async (payload:
unknown) => {        // Gate 1: Payload validation        const result =
MoveSchema.safeParse(payload);        if (!result.success)
{            socket.emit('ERROR', { code: 'INVALID_PAYLOAD', event:
'PLAYER_MOVE' });            return;        }        const { x, y, z, roomId } =
result.data;        // Gate 2: Room authorization        if
(socket.data.currentRoomId !== roomId) {            socket.emit('ERROR', { code:
'ROOM_MISMATCH' });            incrementViolation(socket);            return;        }
// Gate 3: Speed validation (server-authoritative)        const now =
Date.now();        const elapsed = (now - lastMoveTime) / 1000; // seconds
const distance = Math.sqrt(            Math.pow(x - lastPosition.x, 2) +
Math.pow(y - lastPosition.y, 2) +            Math.pow(z - lastPosition.z,
2),        );        const maxAllowedDistance = MAX_SPEED * elapsed;        if
(distance > maxAllowedDistance * SPEED_HACK_MULTIPLIER)
{            console.warn({                event: 'SPEED_HACK_DETECTED',                userId:
socket.data.user.id,                distance,                maxAllowed:
maxAllowedDistance,            });            socket.emit('POSITION_CORRECTION',
lastPosition); // Snap back            incrementViolation(socket);            return;
}        // Update server-authoritative position        lastPosition = { x, y,
z };        lastMoveTime = now;        // Broadcast server-validated position (not
client's raw payload)        socket.to(roomId).emit('PLAYER_MOVED',
{            userId: socket.data.user.id,            x, y, z,        });    }); } function
incrementViolation(socket: Socket): void {    socket.data.violations =
(socket.data.violations ?? 0) + 1;    if (socket.data.violations >= 3)
{        socket.emit('DISCONNECTED', { reason: 'ANTI_CHEAT_VIOLATION' });
socket.disconnect(true);    } }

```

## 5.4 Message Size Limits & Rate Limiting

```

// src/server/socket/rateLimiter.ts // Per-client WebSocket event rate
limiting using Redis import { redis } from '../redis.js'; import type
{ Socket } from 'socket.io'; interface RateLimitConfig {    maxEvents:
number;    windowSeconds: number; } const LIMITS: Record<string,
RateLimitConfig> = {    SEND_CHAT:    { maxEvents: 10,    windowSeconds: 5    },
PLAYER_MOVE:    { maxEvents: 100, windowSeconds: 5    },    // ~20/sec
START_FISHING: { maxEvents: 1,    windowSeconds: 5    },    RING_DOORBELL:
{ maxEvents: 3,    windowSeconds: 60 },    TRADE_INITIATE:{ maxEvents: 3,
windowSeconds: 30 }, }; export async function
checkSocketRateLimit(    socket: Socket,    event: string, ): Promise<boolean>
{    const config = LIMITS[event];    if (!config) return true; // No limit
defined for this event    const key = `ratelimit:socket:${socket.id}:$
{event}`;    const current = await redis.incr(key);    if (current === 1) {
// First increment – set expiry on the key    await redis.expire(key,
config.windowSeconds);    }    if (current > config.maxEvents)
{        socket.emit('RATE_LIMITED', { event, retryAfter:
config.windowSeconds });        // Track rate limit violations
socket.data.violations = (socket.data.violations ?? 0) + 1;        if
(socket.data.violations >= 3) {            socket.emit('DISCONNECTED', { reason:
'RATE_LIMIT_EXCEEDED' });            socket.disconnect(true);        }        return
false; // Caller should return early    }    return true; // OK to proceed }
// Usage in event handler: // socket.on('SEND_CHAT', async (payload) => { //
if (!await checkSocketRateLimit(socket, 'SEND_CHAT')) return; //    // ...
process chat // });

```

## 5.5 Idle Connection Management & Ping/Pong

```

// src/server/socket/idleTimeout.ts import type { Socket } from 'socket.io';
const IDLE_TIMEOUT_MS = 10 * 60 * 1000; // 10 minutes export function

```

```

idleTimeoutMiddleware(socket: Socket, next: () => void): void { let
idleTimer: NodeJS.Timeout; const resetTimer = () =>
{ clearTimeout(idleTimer); idleTimer = setTimeout(() =>
{ console.info({ event: 'IDLE_DISCONNECT', userId:
socket.data.user?.id, timestamp: new Date().toISOString(), });
socket.emit('DISCONNECTED', { reason: 'IDLE_TIMEOUT' });
socket.disconnect(true); }, IDLE_TIMEOUT_MS); }; // Track activity
on any incoming event socket.onAny(() => resetTimer()); // Start the
timer resetTimer(); // Clean up on disconnect socket.on('disconnect',
() => clearTimeout(idleTimer)); next(); }

```

## 5.6 Room Authorization Enforcement

```

// src/server/socket/handlers/roomHandler.ts import { prisma } from
'../../prisma.js'; import type { Socket, Server } from 'socket.io'; import
{ JoinRoomSchema } from '../../validation/schemas.js'; export function
registerRoomHandler(socket: Socket, io: Server): void
{ socket.on('JOIN_ROOM', async (payload: unknown) => { const result =
JoinRoomSchema.safeParse(payload); if (!result.success)
{ socket.emit('ERROR', { code: 'INVALID_PAYLOAD' });
return; } const { roomId } = result.data; const userId =
socket.data.user.id; // Always read room state from DATABASE – never
trust client-sent access mode const room = await prisma.room.findUnique({
where: { id: roomId }, select: { id: true, accessMode: true, ownerId:
true, isActive: true }, }); if (!room || !room.isActive)
{ socket.emit('ERROR', { code: 'ROOM_NOT_FOUND' }); return; }
// Private loft access check if (room.accessMode === 'PRIVATE')
{ const isOwner = room.ownerId === userId; const isInvited =
await prisma.loftAccess.findUnique({ where: { roomId_userId:
{ roomId, userId }, }, }); if (!isOwner && !isInvited)
{ socket.emit('ERROR', { code: 'ACCESS_DENIED' }); return;
} } // Leave previous room if (socket.data.currentRoomId) {
socket.leave(socket.data.currentRoomId);
socket.to(socket.data.currentRoomId).emit('PLAYER_LEFT', { userId }); }
// Join new room – set currentRoomId SERVER-SIDE only
socket.data.currentRoomId = roomId; socket.join(roomId); // Do NOT
enumerate all rooms – only return rooms with active users const
usersInRoom = await io.in(roomId).fetchSockets();
socket.emit('ROOM_JOINED', { roomId, userCount:
usersInRoom.length, }); socket.to(roomId).emit('PLAYER_JOINED',
{ userId, username: socket.data.user.username }); }); }

```

---

## 6. JWT Security MAC

### 6.1 Token Configuration Audit

```

# Generate a cryptographically secure JWT_SECRET (run once, store
in .env.production) # [MAC / LINUX] node -e
"console.log(require('crypto').randomBytes(32).toString('hex'))" # Example
output: a7f3c2e8b1d94e6f5a0c3b7e2d8f1a4c9e6b3d0f7a2c5e8b1d4f7a0c3b6e9d2f #
Add to /etc/environment on the server (or your .env.production) echo
"JWT_SECRET=a7f3c2e8b1d94e6f5a0c3b7e2d8f1a4c9e6b3d0f7a2c5e8b1d4f7a0c3b6e9d2f"
| \ sudo tee -a /etc/environment

```

```
// src/server/auth/tokens.ts import jwt from 'jsonwebtoken'; import crypto
from 'crypto'; import { prisma } from '../prisma.js'; const JWT_SECRET =
process.env.JWT_SECRET!; // Startup assert already ran const
ACCESS_TOKEN_EXPIRY = '15m'; const REFRESH_TOKEN_EXPIRY = '7d'; const
REFRESH_TOKEN_EXPIRY_MS = 7 * 24 * 60 * 60 * 1000; export function
generateAccessToken(userId: string, username: string): string { return
jwt.sign(    { userId, username },    JWT_SECRET,    {    algorithm:
'HS256',    // Explicitly set algorithm    expiresIn: ACCESS_TOKEN_EXPIRY,
issuer: 'havenworld-api',    audience: 'havenworld-client',    },    ); }
export function verifyAccessToken(token: string): { userId: string; username:
string } {    // Algorithm pinning – REQUIRED to prevent algorithm confusion
attacks    // Without this, an attacker could craft a token with alg: "none"
return jwt.verify(token, JWT_SECRET, {    algorithms: ['HS256'],    issuer:
'havenworld-api',    audience: 'havenworld-client',    }) as { userId:
string; username: string }; } export async function
generateRefreshToken(userId: string): Promise<string> {    const token =
crypto.randomBytes(48).toString('base64url');    const expiresAt = new
Date(Date.now() + REFRESH_TOKEN_EXPIRY_MS);    await
prisma.refreshToken.create({    data: {    token,    userId,
expiresAt,    // familyId groups token chains for reuse detection
familyId: crypto.randomUUID(),    },    });    return token; }
```

## 6.2 Refresh Token Rotation

```
// src/server/routes/auth.ts – /api/auth/refresh endpoint import { prisma }
from '../prisma.js'; import { generateAccessToken, generateRefreshToken }
from '../auth/tokens.js'; export async function refreshHandler(req, res) {
const incomingToken = req.cookies?.refresh_token;    if (!incomingToken) {
return res.status(401).json({ error: 'REFRESH_TOKEN_MISSING' });    }    const
storedToken = await prisma.refreshToken.findUnique({    where: { token:
incomingToken },    include: { user: { select: { id: true, username: true,
isBanned: true } } },    });    // Token not found (never issued or already
rotated)    if (!storedToken) {    // This is suspicious – may indicate token
reuse attack    // Try to find the family this token belonged to and revoke
all    console.warn({    event: 'REFRESH_TOKEN_REUSE_SUSPECTED',
token: incomingToken.substring(0, 8) + '...',    // Log only prefix
timestamp: new Date().toISOString(),    });    return
res.status(401).json({ error: 'TOKEN_REUSE_DETECTED' });    }    // Token
expired    if (storedToken.expiresAt < new Date()) {    await
prisma.refreshToken.delete({ where: { id: storedToken.id } });    return
res.status(401).json({ error: 'REFRESH_TOKEN_EXPIRED' });    }    // Check if
user was banned since token was issued    if (storedToken.user.isBanned) {
await prisma.refreshToken.deleteMany({ where: { userId:
storedToken.userId } });    return res.status(403).json({ error:
'ACCOUNT_BANNED' });    }    // — Rotation: invalidate old token, issue new
one atomically    const [newRefreshToken] = await prisma.
$transaction(async (tx) => {    // Delete the used token    await
tx.refreshToken.delete({ where: { id: storedToken.id } });    // Issue a
new refresh token    const newToken = await
generateRefreshToken(storedToken.userId);    return [newToken];    });
const newAccessToken = generateAccessToken(    storedToken.user.id,
storedToken.user.username,    );    // Set new refresh token cookie
res.cookie('refresh_token', newRefreshToken, {    httpOnly: true,
secure: true,    sameSite: 'strict',    maxAge: 7 * 24 * 60 * 60 * 1000,
path: '/api/auth',    });    res.json({ accessToken: newAccessToken }); }
```

## 6.3 Token Storage Security

| Token Type         | Storage Location                                                                 | Rationale                                                               | Never Store In                               |
|--------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------------|----------------------------------------------|
| Access Token (JWT) | JavaScript memory (module-level variable, not persisted)                         | Expires in 15 min; memory storage prevents XSS theft                    | localStorage, sessionStorage, regular cookie |
| Refresh Token      | httpOnly, SameSite=Strict, Secure cookie; also in PostgreSQL RefreshTokens table | httpOnly prevents XSS; DB storage enables revocation and audit          | JavaScript memory, localStorage, JWT payload |
| CSRF Token         | Non-httpOnly cookie (JavaScript-readable) + request header                       | Double-submit pattern; cross-site attacker cannot read the cookie value | JWT payload, server-side session only        |

## 6.4 Logout & Session Termination

```
// src/server/routes/auth.ts – Logout endpoints // Single device logout
export async function logoutHandler(req, res) {  const refreshToken =
req.cookies?.refresh_token;  if (refreshToken) {    // Delete from DB
(invalidates it for refresh rotation)    await
prisma.refreshToken.deleteMany({ where: { token: refreshToken } });  }  //
Clear the cookie regardless of whether token was in DB
res.clearCookie('refresh_token', { path: '/api/auth' });
res.clearCookie('csrf_token');  res.json({ success: true }); } // Logout
all devices (user-initiated or admin-forced) export async function
logoutAllHandler(req, res) {  const userId = req.user.id; // Set by auth
middleware  // Delete ALL refresh tokens for this user  const { count } =
await prisma.refreshToken.deleteMany({ where: { userId } });
console.info({    event: 'LOGOUT_ALL_DEVICES',    userId,
tokensRevoked: count,    timestamp: new Date().toISOString(),  });
res.clearCookie('refresh_token', { path: '/api/auth' });
res.clearCookie('csrf_token');  res.json({ success: true, devicesLoggedOut:
count }); } // Admin: force-logout a specific user (ban enforcement) export
async function adminForceLogoutHandler(req, res) {  const { targetUserId } =
req.params;  await prisma.
$action([    prisma.refreshToken.deleteMany({ where: { userId:
targetUserId } })),    prisma.user.update({      where: { id:
targetUserId },      data: { isBanned: true, bannedAt: new Date() },    }),
]);  // The target user's next API call or refresh attempt will receive 403
ACCOUNT_BANNED  res.json({ success: true }); }
```

## 7. Game Logic Security MAC LINUX

### 7.1 Server-Authoritative Movement HIGH

Client-sent positions must be treated as movement *intent*, not as ground truth. The server computes the actual position, validates it against the navmesh and speed limit, and broadcasts its own authoritative result to all players in the room. This prevents teleportation, speed hacks, and wall-clip exploits.

```
// src/server/game/movement.ts interface Position { x: number; y: number; z: number; } const MAX_SPEED_UNITS_PER_SECOND = 5; const SPEED_HACK_TOLERANCE = 2.0; // Allow up to 2x max speed before flagging export function validateMovement( currentPos: Position, requestedPos: Position, elapsedMs: number, ): { valid: boolean; reason?: string; clampedPos?: Position } { const elapsedSeconds = Math.max(elapsedMs / 1000, 0.016); // At least one frame const distance = Math.sqrt( Math.pow(requestedPos.x - currentPos.x, 2) + Math.pow(requestedPos.y - currentPos.y, 2) + Math.pow(requestedPos.z - currentPos.z, 2), ); const maxAllowed = MAX_SPEED_UNITS_PER_SECOND * elapsedSeconds; if (distance > maxAllowed * SPEED_HACK_TOLERANCE) { return { valid: false, reason: 'SPEED_EXCEEDED' }; } // TODO (Part 9): Integrate navmesh validation here // const isWalkable = navmesh.isPointWalkable(requestedPos); // if (!isWalkable) return { valid: false, reason: 'OFF_NAVMESH' }; // Clamp to max allowed distance in the requested direction if (distance > maxAllowed) { const ratio = maxAllowed / distance; return { valid: true, clampedPos: { x: currentPos.x + (requestedPos.x - currentPos.x) * ratio, y: currentPos.y + (requestedPos.y - currentPos.y) * ratio, z: currentPos.z + (requestedPos.z - currentPos.z) * ratio, }, }; } return { valid: true, clampedPos: requestedPos }; }
```

### 7.2 Economy Exploit Prevention HIGH

#### Security Critical: All Economy Writes Must Be Transactional

Never read a user's balance, check it, and then update it in separate database calls. Between the read and the write, another concurrent request could modify the balance, creating a race condition that allows duplication exploits. All economy operations must use Prisma's `$transaction` to guarantee atomicity.

```
// src/server/game/economy.ts import { prisma } from '../prisma.js'; export async function transferCoins( senderId: string, receiverId: string, amount: number, ): Promise<{ success: boolean; error?: string }> { // Validate amount is a positive integer if (!Number.isInteger(amount) || amount <= 0) { return { success: false, error: 'INVALID_AMOUNT' }; } // Prevent self-transfer if (senderId === receiverId) { return { success: false, error: 'SELF_TRANSFER_FORBIDDEN' }; } try { await
```



```

prisma.$transaction(async (tx) => {      // Read sender balance INSIDE the
transaction (atomic read)      const sender = await tx.user.findUnique({
where: { id: senderId },      select: { id: true, coins: true,
weeklyEarnings: true },      });      if (!sender) throw new
Error('SENDER_NOT_FOUND');      // Verify sufficient balance INSIDE the
transaction      if (sender.coins < amount) throw new
Error('INSUFFICIENT_BALANCE');      // Deduct from sender      await
tx.user.update({      where: { id: senderId },      data: { coins:
{ decrement: amount } },      });      // Credit receiver      await
tx.user.update({      where: { id: receiverId },      data: { coins:
{ increment: amount } },      });      // Audit log – every coin movement
recorded      await tx.transactionLog.create({      data:
{      senderId,      receiverId,      amount,      type:
'Transfer',      timestamp: new Date(),      },      });      });
return { success: true }; } catch (err) {      const message = (err as
Error).message;      return { success: false, error: message }; } } //
Weekly earnings cap enforcement export async function awardReward(  userId:
string,  amount: number,  source: string, ): Promise<{ awarded: number;
capReached: boolean }> {  const WEEKLY_CAP = 500;  // Coins  return await
prisma.$transaction(async (tx) => {      const user = await
tx.user.findUnique({      where: { id: userId },      select:
{ weeklyEarnings: true },      });      if (!user) throw new
Error('USER_NOT_FOUND');      const remaining = Math.max(0, WEEKLY_CAP -
user.weeklyEarnings);      const awarded = Math.min(amount, remaining);
if (awarded > 0) {      await tx.user.update({      where: { id:
userId },      data: {      coins:      { increment: awarded },
weeklyEarnings: { increment: awarded },      },      });      await
tx.transactionLog.create({      data: { receiverId: userId, amount:
awarded, type: 'REWARD', source },      });      }      return { awarded,
capReached: remaining === 0 }; } }); }

```

## 7.3 P2P Trade Security ● HIGH

```

// src/server/game/trade.ts // Trade state machine: PENDING → LOCKED →
CONFIRMED (atomic swap) | CANCELLED import { prisma } from '../prisma.js';
type TradeState = 'PENDING' | 'LOCKED' | 'CONFIRMED' | 'CANCELLED'; const
VALID_TRANSITIONS: Record<TradeState, TradeState[]> = {  PENDING:
['LOCKED', 'CANCELLED'],  LOCKED:  ['CONFIRMED', 'CANCELLED'],
CONFIRMED: [],  // Terminal state  CANCELLED: [],  // Terminal state };
export async function confirmTrade(tradeId: string, userId: string)
{  return await prisma.$transaction(async (tx) => {      // Lock the trade
row for this transaction (prevents concurrent confirms)      const trade =
await tx.trade.findUnique({      where: { id: tradeId },      include: {
initiator: { select: { id: true, coins: true } },      target:
{ select: { id: true, coins: true } },      offeredItems: true,
requestedItems: true,      },      });      if (!trade) throw new
Error('TRADE_NOT_FOUND');      if (trade.state !== 'LOCKED') throw new
Error('INVALID_TRADE_STATE');      if (trade.initiatorId !== userId &&
trade.targetId !== userId) {      throw new Error('NOT_TRADE_PARTICIPANT');
}      // Check trade has not timed out (120 second window)      const ageMs =
Date.now() - trade.updatedAt.getTime();      if (ageMs > 120_000)
{      await tx.trade.update({ where: { id: tradeId }, data: { state:
'CANCELLED' } });      throw new Error('TRADE_EXPIRED');      }      //
Verify all offered items are still in initiator's inventory      for (const
item of trade.offeredItems) {      const owned = await
tx.inventoryItem.findUnique({      where: { id_ownerId: { id: item.id,
ownerId: trade.initiatorId } },      });      if (!owned) throw new
Error(`ITEM_NOT_IN_INVENTORY: ${item.id}`);      }      // Atomic item swap:
reassign ownership of all items in one transaction      await

```

```
tx.inventoryItem.updateMany({      where: { id: { in:
trade.offeredItems.map((i) => i.id) } },      data: { ownerId:
trade.targetId },      });      await
tx.inventoryItem.updateMany({      where: { id: { in:
trade.requestedItems.map((i) => i.id) } },      data: { ownerId:
trade.initiatorId },      });      await tx.trade.update({ where: { id:
tradeId }, data: { state: 'CONFIRMED' } });      }); }
```

## 7.4 Fishing Anti-Cheat 🟡 MEDIUM

```
// src/server/game/fishing.ts // Server generates catch result – client
CANNOT specify fish type or rarity import crypto from 'crypto'; import
{ prisma } from '../prisma.js'; interface FishingSession {  userId: string;
roomId: string;  spotId: string;  startTime: number;  sessionKey:
string; } // In-memory store for active fishing sessions (use Redis in
production at scale) const activeSessions = new Map<string,
FishingSession>(); // Starts a fishing session – returns a session key to
the client export function startFishingSession(userId: string, roomId:
string, spotId: string): string {  const sessionKey =
crypto.randomBytes(16).toString('hex');  activeSessions.set(userId,
{    userId, roomId, spotId,    startTime: Date.now(),
sessionKey,  });  return sessionKey; } // Validates timing and generates
server-authoritative catch result export async function processFishingResult(
userId: string,  tensionValue: number, // 0.0-1.0 from client mini-game
clientSessionKey: string, ) {  const session = activeSessions.get(userId);
if (!session) throw new Error('NO_ACTIVE_FISHING_SESSION');  if
(session.sessionKey !== clientSessionKey) throw new
Error('SESSION_KEY_MISMATCH');  const elapsed = Date.now() -
session.startTime;  if (elapsed < 3000 || elapsed > 30000) {    throw new
Error('INVALID_CAST_TIMING'); // Too fast (bot) or too slow (timeout)  }
activeSessions.delete(userId); // Server-generated result – client has no
influence over fish type  const rand = crypto.randomInt(0, 100);  let
fishType: string;  let coinReward: number; // Higher tension + better
timing improves probabilities (server-computed)  const successScore =
tensionValue * 0.6 + (1 - Math.abs(elapsed - 8000) / 22000) * 0.4;  if
(successScore > 0.85 && rand < 5)    { fishType = 'RARE';    coinReward =
20; }  else if (successScore > 0.60 && rand < 30) { fishType = 'UNCOMMON';
coinReward = 10; }  else if (rand < 70)    { fishType
= 'COMMON';    coinReward = 5; }  else
{ fishType = 'NOTHING'; coinReward = 0; }  if (coinReward > 0) {    await
awardReward(userId, coinReward, `FISHING_${fishType}`);  }  return
{ fishType, coinReward }; }
```

## 7.5 Invite Code Security 🟡 MEDIUM

```
// src/server/game/inviteCodes.ts import crypto from 'crypto'; import
{ prisma } from '../prisma.js'; export async function
generateInviteCode(adminId: string): Promise<string> {  // Cryptographically
random, URL-safe  const code = crypto.randomBytes(6).toString('base64url');
// 8-char URL-safe string  await prisma.inviteCode.create({    data: {
code,    createdById: adminId,    expiresAt: new Date(Date.now() + 30 *
24 * 60 * 60 * 1000), // 30 days    isUsed: false,  },  });  return
code; } export async function redeemInviteCode(code: string, newUserId:
string): Promise<void> {  // Atomic redemption – marks as used in the same
operation as user creation  // (user creation and invite redemption happen
in the same Prisma $transaction  // in the registration handler)  await
prisma.$transaction(async (tx) => {    const invite = await
tx.inviteCode.findUnique({ where: { code } });    if (!invite) throw new
Error('INVITE_CODE_INVALID');    if (invite.isUsed) throw new
```

```
Error('INVITE_CODE_ALREADY_USED');    if (invite.expiresAt < new Date())
throw new Error('INVITE_CODE_EXPIRED');    await tx.inviteCode.update({
where: { code },    data: { isUsed: true, usedById: newUserId, usedAt: new
Date() },    });    }); }
# Cron job to expire old invite codes daily at 2am [LINUX] sudo crontab -e #
Add: 0 2 * * * psql -U havenworld_user -d havenworld_prod \ -c "UPDATE
invite_codes SET is_expired = true WHERE expires_at < NOW() AND is_used =
false;"
```

## 7.6 Admin Privilege Security ● HIGH

```
// src/server/middleware/adminAuth.ts import { prisma } from '../prisma.js';
import type { Request, Response, NextFunction } from 'express'; // Admin
role is ALWAYS read from the database – never trusted from JWT // This
ensures banning, demotion, or role changes take effect immediately // without
waiting for token expiry export async function requireAdmin( req: Request,
res: Response, next: NextFunction, ): Promise<void> { const userId =
req.user?.id; // Set by the auth middleware chain if (!userId)
{ res.status(401).json({ error: 'AUTH_REQUIRED' }); return; } //
Fresh DB read on every admin request – no caching of role const user =
await prisma.user.findUnique({ where: { id: userId }, select: { role:
true, isBanned: true }, }); if (!user || user.isBanned || user.role !==
'admin') { res.status(403).json({ error: 'ADMIN_REQUIRED' }); return;
} next(); } // Admin action logger – wrap all admin handlers with this
export function logAdminAction(action: string) { return async (req:
Request, res: Response, next: NextFunction) => { // Log before action
executes await prisma.adminAuditLog.create({ data:
{ adminId: req.user!.id, action, targetId:
req.params.userId ?? req.body?.targetUserId ?? null, ip:
req.ip ?? 'unknown', userAgent: req.get('user-agent') ?? 'unknown',
timestamp: new Date(), }, }); next(); }; } // Route
registration: // router.post('/admin/ban/:userId', // requireAuth, //
requireAdmin, // logAdminAction('BAN_USER'), // banUserHandler, // );
```

# 8. Database Security LINUX

## 8.1 PostgreSQL Hardening

```
# Connect as the postgres superuser sudo -u postgres psql # Create dedicated
application user with minimal permissions CREATE USER havenworld_user WITH
PASSWORD 'use_a_strong_random_password_here'; CREATE DATABASE havenworld_prod
OWNER havenworld_user; # Grant only the permissions the app needs – no
CREATE, no DROP GRANT CONNECT ON DATABASE havenworld_prod TO havenworld_user;
GRANT USAGE ON SCHEMA public TO havenworld_user; GRANT SELECT, INSERT,
UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO havenworld_user; GRANT
USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO havenworld_user; ALTER
DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT, INSERT, UPDATE, DELETE ON
TABLES TO havenworld_user; ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT
USAGE, SELECT ON SEQUENCES TO havenworld_user; # Verify: havenworld_user
cannot CREATE or DROP tables \c havenworld_prod havenworld_user CREATE TABLE
test_unauthorized (id serial); -- Should fail with permission denied
# /etc/postgresql/16/main/pg_hba.conf – access control # TYPE DATABASE
USER ADDRESS METHOD # Local superuser – peer
authentication (OS user postgres only) local all postgres
peer # Application user – local TCP only, md5 password host
```

```

havenworld_prod    havenworld_user    127.0.0.1/32    md5    # Deny all remote
connections (no external PostgreSQL access) # host    all    all
0.0.0.0/0          reject    # <- data-id="581" This is the default if no rule
matches
# /etc/postgresql/16/main/postgresql.conf – key security settings
listen_addresses = 'localhost'          # Bind to localhost only – no external
exposure max_connections = 100           # Limit concurrent connections
statement_timeout = '30s'                # Kill runaway queries after 30 seconds
idle_in_transaction_session_timeout = '60s' # Kill stuck transactions
log_connections = on                     # Log all new connections
log_disconnections = on log_duration = off # Don't log
query durations (PII risk) log_statement = 'none' # Don't log
SQL statements in production (PII risk)
# Set to 'ddl' temporarily for debugging schema migrations
# Reload configuration sudo systemctl reload postgresql # Verify pg_hba
settings took effect sudo -u postgres psql -c "SELECT type, database,
user_name, address, auth_method FROM pg_hba_file_rules;"

```

## 8.2 Redis Hardening

```

# Generate a strong Redis password node -e
"console.log(require('crypto').randomBytes(24).toString('hex'))" #
/etc/redis/redis.conf – security settings bind 127.0.0.1          #
Localhost only – no external exposure port 6379 protected-mode yes # Require
authentication requirepass YOUR_STRONG_REDIS_PASSWORD_HERE # Disable
dangerous commands rename-command CONFIG "" rename-command FLUSHALL ""
rename-command FLUSHDB "" rename-command KEYS "" rename-command DEBUG
"" rename-command SLAVEOF "" # Memory management – prevent OOM crashes
maxmemory 256mb maxmemory-policy allkeys-lru # Persistence – save every 15
min if at least 1 key changed save 900 1 save 300 10 # Logging loglevel
notice logfile /var/log/redis/redis-server.log
# Restart Redis and verify sudo systemctl restart redis-server # Test
authentication required redis-cli ping # NOAUTH Authentication required
redis-cli -a YOUR_STRONG_REDIS_PASSWORD_HERE ping # PONG # Verify localhost
binding (6379 should NOT appear on 0.0.0.0) sudo ss -tlnp | grep 6379 #
Should show: 127.0.0.1:6379 only

```

## 8.3 Backup Security

```

# /opt/havenworld/scripts/backup.sh – Daily encrypted backup script
#!/bin/bash set -euo pipefail BACKUP_DIR="/var/backups/havenworld" DATE=$(
date +%Y-%m-%d_%H-%M) FILENAME="havenworld_${DATE}.sql.gz"
GPG_RECIPIENT="your-gpg-key-id-or-email" # Create backup directory if not
exists mkdir -p "$BACKUP_DIR" # Dump database and compress
PGPASSWORD="$DB_PASSWORD" pg_dump \ -h 127.0.0.1 \ -U havenworld_user \
-d havenworld_prod \ --no-owner \ --no-acl \ | gzip > "${BACKUP_DIR}/${
FILENAME}" # Encrypt with GPG (symmetric, AES256) gpg --batch \ --yes \
--symmetric \ --cipher-algo AES256 \ --passphrase-file
/etc/havenworld/backup-passphrase \ "${BACKUP_DIR}/${FILENAME}" # Remove
unencrypted backup immediately rm "${BACKUP_DIR}/${FILENAME}" echo "[($
date)] Backup completed: ${FILENAME}.gpg" # Prune backups older than 7 days
find "$BACKUP_DIR" -name "*.sql.gz.gpg" -mtime +7 -delete # Log completion
logger -t havenworld-backup "Backup ${FILENAME}.gpg created and encrypted"
# Add to root crontab – runs at 3:00 AM daily sudo crontab -e # Add: 0 3 * *
* /opt/havenworld/scripts/backup.sh >> /var/log/havenworld-backup.log 2>&1 #
Store GPG passphrase securely sudo mkdir -p /etc/havenworld echo "your-very-
strong-passphrase-here" | sudo tee /etc/havenworld/backup-passphrase sudo
chmod 600 /etc/havenworld/backup-passphrase sudo chown root:root
/etc/havenworld/backup-passphrase

```

```
# Test restore procedure (do this quarterly) # Decrypt backup gpg --batch --
passphrase-file /etc/havenworld/backup-passphrase \      --output
/tmp/restore_test.sql.gz \      --decrypt
/var/backups/havenworld/havenworld_YYYY-MM-DD_03-00.sql.gz.gpg # Create
restore DB (do NOT restore over production) sudo -u postgres createdb
havenworld_restore # Restore gunzip -c /tmp/restore_test.sql.gz | \
PGPASSWORD="$DB_PASSWORD" psql -h 127.0.0.1 -U havenworld_user
havenworld_restore # Verify key tables exist and have data
PGPASSWORD="$DB_PASSWORD" psql -h 127.0.0.1 -U havenworld_user
havenworld_restore \ -c "SELECT COUNT(*) FROM users; SELECT COUNT(*) FROM
rooms;" # Clean up sudo -u postgres dropdb havenworld_restore rm
/tmp/restore_test.sql.gz
```

## 9. Dependency Security MAC

### 9.1 npm audit Workflow

```
# Run after every npm install npm audit # In CI – fail the build on HIGH or
CRITICAL severity vulnerabilities npm audit --audit-level=high # Auto-fix
safe updates (patch versions, non-breaking) npm audit fix # Review what
audit fix would change before applying npm audit fix --dry-run # For complex
vulnerabilities requiring manual intervention: # 1. Read the advisory URL in
npm audit output # 2. Check if a fixed version exists: npm info vulnerable-
package versions # 3. Force pin to fixed version: npm install vulnerable-
package@x.y.z # 4. If no fix: npm install --save-dev patch-package → patch
the vulnerability locally
```

**Add to .github/workflows/pr-check.yml:**

```
name: PR Security Check on: pull_request: branches: [main] jobs:
security-audit: name: npm Audit runs-on: ubuntu-latest steps:
- uses: actions/checkout@v4 - name: Setup Node.js uses:
actions/setup-node@v4 with: node-version: '22'
cache: 'npm' - name: Install dependencies run: npm ci -
name: Run npm audit (fail on high+) run: npm audit --audit-level=high
- name: Run Snyk security scan uses: snyk/actions/node@master
env: SNYK_TOKEN: ${ secrets.SNYK_TOKEN } with:
args: --severity-threshold=high - name: License compliance check
run: npm run check-licenses
```

### 9.2 Dependabot Setup

```
# .github/dependabot.yml version: 2 updates: - package-ecosystem: "npm"
directory: "/" schedule: interval: "weekly" day: "monday"
time: "09:00" timezone: "America/Chicago" open-pull-requests-limit:
5 # Auto-merge patch updates (low risk) # Review minor and major
updates manually groups: patch-updates: update-types:
- "patch" reviewers: - "dboone323" labels: -
"dependencies" - "security" commit-message: prefix: "deps"
prefix-development: "dev-deps" include: "scope" ignore: # Pin
major version of Socket.io until Part 9 (compatibility review) -
dependency-name: "socket.io" update-types: ["version-update:semver-
major"] - package-ecosystem: "github-actions" directory: "/"
schedule: interval: "weekly"
```

## 9.3 GPL-3.0 License Compliance

```
# Install license checker npm install --save-dev license-checker # Add to
package.json scripts { "scripts": { "check-licenses": "license-checker
--production --failOn 'AGPL-3.0;UNLICENSED;UNKNOWN;CC-BY-NC'" } }
```

| License      | GPL-3.0 Compatible?  | Action Required                                          |
|--------------|----------------------|----------------------------------------------------------|
| MIT          | ✓ Yes                | None — freely combine                                    |
| BSD-2-Clause | ✓ Yes                | None                                                     |
| BSD-3-Clause | ✓ Yes                | None                                                     |
| Apache-2.0   | ✓ Yes (GPL-3.0 only) | None for GPL-3.0; note in NOTICE file                    |
| ISC          | ✓ Yes                | None                                                     |
| LGPL-2.1     | ✓ Compatible         | Document the dependency; do not statically link          |
| GPL-3.0      | ✓ Same               | None                                                     |
| AGPL-3.0     | ✗ Problematic        | Find alternative or document exception with legal review |
| UNLICENSED   | ✗ Reject             | Find alternative — no usage rights granted               |
| CC-BY-NC     | ✗ Reject             | Non-commercial restriction conflicts with distribution   |

---

## 10. Monitoring & Incident Response LINUX

### 10.1 Sentry Configuration for Security Events

```
// src/server/monitoring/sentry.ts import * as Sentry from '@sentry/node';
export function initializeSentry(): void {  Sentry.init({    dsn:
process.env.SENTRY_DSN,    environment: process.env.NODE_ENV,
tracesSampleRate: process.env.NODE_ENV === 'production' ? 0.1 : 1.0,    //
— PII Scrubbing — NEVER send tokens or passwords to Sentry —
beforeSend(event) {    // Scrub request bodies that may contain
credentials    if (event.request?.data) {    const data =
event.request.data as Record<string, unknown>;    if (data.password)
data.password = '[REDACTED]';    if (data.inviteCode)
data.inviteCode = '[REDACTED]';    if (data.refreshToken)
data.refreshToken = '[REDACTED]';    }    // Scrub authorization
headers    if (event.request?.headers) {    if
(event.request.headers['authorization'])
{    event.request.headers['authorization'] = '[REDACTED]';    }
if (event.request.headers['cookie'])
```

```

{      event.request.headers['cookie'] = '[REDACTED]';      }
return event;    },    }); } // Security-specific event capture helpers
export function captureSecurityEvent(  event: string,  data: Record<string,
unknown>,  level: Sentry.SeverityLevel = 'warning', ): void
{  Sentry.withScope((scope) => {    scope.setTag('security.event', event);
scope.setLevel(level);    // Never include tokens, passwords, or session IDs
scope.setContext('security', {      event,      timestamp: new
Date().toISOString(),      ...data,    });
Sentry.captureMessage(`[SECURITY] ${event}`, level);  }); } // Usage
examples: // captureSecurityEvent('AUTH_BRUTE_FORCE', { ip, attempts: 20 },
'error'); // captureSecurityEvent('ORIGIN_FORBIDDEN', { origin, ip },
'warning'); // captureSecurityEvent('ECONOMY_ANOMALY', { userId,
expectedBalance, actualBalance }, 'error'); //
captureSecurityEvent('ADMIN_FROM_NEW_IP', { adminId, ip }, 'warning');

```

## 10.2 Suspicious Activity Detection

| Trigger                    | Threshold                                       | Automated Response                    | Escalation                                 |
|----------------------------|-------------------------------------------------|---------------------------------------|--------------------------------------------|
| Failed logins              | 5 in 5 min from same IP                         | fail2ban temp IP ban (1 hour)         | Sentry alert if rate > 20/min              |
| Anti-cheat violations      | 3 violations per socket session                 | Socket disconnect                     | 5 total → account 24-hour ban              |
| Speed hack detection       | 1 detection                                     | Position correction + violation count | 3 detections → permanent flag              |
| Chat rate limit violations | 10 violations in 1 hour                         | 24-hour mute                          | Moderator alert in Discord                 |
| Economy anomaly            | Balance inconsistency detected post-transaction | Freeze account, log event             | Immediate Sentry ERROR alert + admin email |
| CSWSH attempt (bad Origin) | Any ORIGIN_FORBIDDEN event                      | Reject connection + log IP            | > 10/min → Sentry alert                    |
| Refresh token reuse        | Any reuse detected                              | Revoke all user sessions              | Sentry WARNING + notify user via email     |
| Admin from new IP          | First admin action from unrecognized IP         | Allow but flag                        | Sentry WARNING + log action                |

### Ban escalation policy:

1. **First offense:** 24-hour account ban (automated)
2. **Second offense:** 7-day ban (automated + moderator review)
3. **Third offense:** Permanent ban (admin action required)

4. **Appeal process:** Discord #ban-appeal channel with case ID from TransactionLog or AdminAuditLog

## 10.3 Security Logging Checklist

```
// src/server/monitoring/logger.ts // Structured security logging – what to
log and what NEVER to log type SecurityLogEvent = | 'AUTH_LOGIN_SUCCESS' |
'AUTH_LOGIN_FAILURE' | 'AUTH_LOGOUT' | 'AUTH_REFRESH_SUCCESS' |
'AUTH_REFRESH_FAILURE' | 'AUTH_BRUTE_FORCE' | 'WS_CONNECTED' |
'WS_DISCONNECTED' | 'WS_ORIGIN_BLOCKED' | 'RATE_LIMIT_HIT' |
'ANTI_CHEAT_VIOLATION' | 'ADMIN_ACTION' | 'BAN_TRIGGER' |
'TRADE_FRAUD_PATTERN'; interface SecurityLog { timestamp: string; level:
'info' | 'warn' | 'error'; event: SecurityLogEvent; userId?: string;
// ✅ OK to log ip?: string; // ✅ OK to log details?:
Record<string, unknown>; // ❌ NEVER include: // password, passwordHash,
accessToken, refreshToken, // sessionId (full), cookie values, credit card,
email, real name } export function securityLog(log: SecurityLog): void
{ const entry = { ...log, timestamp: new Date().toISOString(),
service: 'havenworld-api', }; // Console output is captured by systemd
journal / logrotate console[log.level](JSON.stringify(entry)); }
# Nginx access log exclusion – don't log health check endpoint # In
/etc/nginx/sites-available/havenworld: location /api/health { access_log
off; proxy_pass http://127.0.0.1:3000; } # Configure logrotate for
Node.js application logs # /etc/logrotate.d/havenworld
/var/log/havenworld/*.log { daily rotate 7 compress
delaycompress missingok notifempty sharedscripts postrotate
systemctl reload havenworld-api 2>/dev/null || true endsript }
```

---

## 11. Security Testing Tools MAC LINUX

### 11.1 OWASP ZAP (Free) MAC

```
# Install OWASP ZAP on Mac [MAC] brew install --cask owasp-zap # Run
automated baseline scan against local development server # (Start your
Node.js server first: npm run dev) zap-baseline.py -t http://localhost:3000 -
r zap-report.html # If zap-baseline.py is not in PATH, find ZAP
installation: /Applications/ZAP.app/Contents/Java/zap.sh \ -cmd -quickurl
http://localhost:3000 -quickout /tmp/zap-report.html # On Linux [LINUX]
docker run --network=host \ -v $(pwd):/zap/wrk/:rw \
ghcr.io/zaproxy/zaproxy:stable \ zap-baseline.py -t http://localhost:3000 -
r /zap/wrk/zap-report.html # Key things ZAP tests automatically: # - Missing
security headers (X-Frame-Options, CSP, HSTS) # - Information disclosure
(Server header, error messages) # - Cookie security flags (httpOnly, Secure,
SameSite) # - CORS misconfiguration # - Clickjacking vulnerability # -
Insecure redirects
```

### 11.2 npm audit & Snyk (Free Tier) MAC

```
# Install Snyk CLI [MAC / LINUX] npm install -g snyk # Authenticate with
free account (snyk.io) snyk auth # Test project for vulnerabilities (free:
200 tests/month) snyk test # Test only production dependencies (ignore dev
tools) snyk test --production # Set up continuous monitoring (alerts on new
```



```
CVEs) snyk monitor --project-name="havenworld-api" # Test Docker image if
used # snyk container test node:22-alpine # View results in Snyk dashboard:
app.snyk.io
```

## 11.3 wscat — WebSocket Security Testing MAC

```
# Install wscat globally [MAC / LINUX] npm install -g wscat # Test 1:
Connection without auth token – should be rejected wscat -c
"wss://YOUR_IP.nip.io/socket.io/?EIO=4&transport=websocket" # Expected:
Connect error: AUTH_REQUIRED # Test 2: Connection with valid token – should
connect wscat -c "wss://YOUR_IP.nip.io/socket.io/?
EIO=4&transport=websocket" \ --header "Authorization: Bearer
YOUR_VALID_ACCESS_TOKEN" # Expected: Connected (session: ...) # Test 3:
CSWSH – bad Origin header – should be rejected wscat -c
"wss://YOUR_IP.nip.io/socket.io/?EIO=4&transport=websocket" \ --header
"Origin: https://evil-attacker.com" # Expected: Connect error:
ORIGIN_FORBIDDEN # Test 4: Oversized payload (max frame size = 64KB) python3
-c "import sys; sys.stdout.write('A' * 70000)" | \ wscat -c
"wss://YOUR_IP.nip.io/socket.io/?EIO=4&transport=websocket" # Expected:
Connection closed (frame too large) # Test 5: Rate limit – send SEND_CHAT 15
times rapidly # Run in a loop with a script – after 10 in 5 seconds, should
receive RATE_LIMITED # and eventually be disconnected
```

## 11.4 Custom Penetration Test Scripts MAC

```
// security-tests/brute-force.js // Tests that /api/auth/login bans after 5
failures const BASE_URL = 'http://localhost:3000'; async function
testBruteForceProtection() { console.log('Testing brute-force protection on
/api/auth/login...'); let bannedAt = null; for (let i = 1; i <= 15; i++)
{ const res = await fetch(`${BASE_URL}/api/auth/login`, { method:
'POST', headers: { 'Content-Type': 'application/json' }, body:
JSON.stringify({ username: 'testuser', password: 'wrongpassword' }), });
console.log(`Attempt ${i}: HTTP ${res.status}`); if (res.status === 429)
{ bannedAt = i; console.log(`✅ Rate limited at attempt ${i} –
PASS`); break; } } if (!bannedAt) { console.error(`❌
FAIL: Server did not rate-limit after 15 failed login attempts`);
process.exit(1); } } testBruteForceProtection().catch(console.error); //
Run: node security-tests/brute-force.js
// security-tests/economy-exploit.js // Tests that negative coin transfers
are rejected async function testNegativeTransfer(accessToken) { const res
= await fetch('http://localhost:3000/api/economy/transfer', { method:
'POST', headers: { 'Content-Type': 'application/json',
'Authorization': `Bearer ${accessToken}`, }, body: JSON.stringify({
receiverId: 'some-other-user-uuid', amount: -1000, // Negative –
should be rejected }, )); if (res.status === 400)
{ console.log(`✅ Negative transfer properly rejected – PASS`); } else
{ console.error(`❌ FAIL: Server accepted negative transfer (HTTP $
{res.status})`); process.exit(1); } } // Run: node
security-tests/economy-exploit.js
// security-tests/speed-hack.js // Tests that PLAYER_MOVE with physically
impossible speed is rejected // Requires a running Socket.io connection with
valid auth import { io } from 'socket.io-client'; async function
testSpeedHack(token) { const socket = io('http://localhost:3000',
{ auth: { token }, transports: ['websocket'], });
socket.on('connect', () => { console.log('Connected. Joining room...');
socket.emit('JOIN_ROOM', { roomId: 'test-room-uuid' }); });
socket.on('ROOM_JOINED', () => { // Send an impossible movement (1000
units from origin in 16ms) socket.emit('PLAYER_MOVE', { x: 1000, y:
0, z: 0, roomId: 'test-room-uuid', }); setTimeout(() => {
```

```
// Check for POSITION_CORRECTION event (server snapped us back)
console.log('Checking for speed hack detection...');    }, 500);    });
socket.on('POSITION_CORRECTION', (correctedPos) => {    console.log('✓
Speed hack detected and corrected to:', correctedPos, '- PASS');
socket.disconnect();    process.exit(0);    });    socket.on('ERROR', (err)
=> {    if (err.code === 'ROOM_MISMATCH' || err.code === 'INVALID_PAYLOAD')
{    console.log('✓ Properly rejected:', err.code, '- PASS');
socket.disconnect();    process.exit(0);    }    });    setTimeout(() => {
console.error('✗ FAIL: No speed hack rejection received within timeout');
process.exit(1);    }, 5000); } // Run: node security-tests/speed-hack.js
```

## 12. Pre-Alpha Security Checklist

Complete all ● **HIGH** items before any public alpha launch. ● **MEDIUM** items should be completed within the first sprint of alpha. ● **LOW** items are post-launch improvements.

| Security Item                                               | Risk                                           | Status                           | Command to Verify                                                       |
|-------------------------------------------------------------|------------------------------------------------|----------------------------------|-------------------------------------------------------------------------|
| SSH password authentication disabled                        | <span style="color: red;">●</span> <b>HIGH</b> | <input type="checkbox"/> Pending | <code>grep PasswordAuthenticat<br/>ion /etc/ssh/sshd_conf<br/>ig</code> |
| ED25519 SSH key in use (not RSA)                            | <span style="color: red;">●</span> <b>HIGH</b> | <input type="checkbox"/> Pending | <code>ssh-keygen -l -f ~/<br/>.ssh/havenworld_e<br/>d25519</code>       |
| Root SSH login disabled                                     | <span style="color: red;">●</span> <b>HIGH</b> | <input type="checkbox"/> Pending | <code>grep PermitRootLogin<br/>/etc/ssh/sshd_conf<br/>ig</code>         |
| fail2ban active and configured                              | <span style="color: red;">●</span> <b>HIGH</b> | <input type="checkbox"/> Pending | <code>sudo fail2ban-<br/>client status</code>                           |
| Unattended security updates enabled                         | <span style="color: red;">●</span> <b>HIGH</b> | <input type="checkbox"/> Pending | <code>sudo systemctl<br/>status unattended-<br/>upgrades</code>         |
| UFW firewall enabled, only 22/80/443 open                   | <span style="color: red;">●</span> <b>HIGH</b> | <input type="checkbox"/> Pending | <code>sudo ufw status<br/>verbose</code>                                |
| Nginx security headers present (HSTS, CSP, X-Frame-Options) | <span style="color: red;">●</span> <b>HIGH</b> | <input type="checkbox"/> Pending | <code>curl -I https://YOUR_IP.nip<br/>.io</code>                        |
| TLS 1.2/1.3 only — TLS 1.0/1.1 disabled                     | <span style="color: red;">●</span> <b>HIGH</b> | <input type="checkbox"/> Pending | <code>curl --tlsv1.0 https://YOUR_IP.nip<br/>.io (should fail)</code>   |

| Security Item                                     | Risk                                                                                     | Status                           | Command to Verify                                                                                  |
|---------------------------------------------------|------------------------------------------------------------------------------------------|----------------------------------|----------------------------------------------------------------------------------------------------|
| OCSP stapling enabled                             |  MEDIUM | <input type="checkbox"/> Pending | <code>openssl s_client -connect YOUR_IP:443 -status</code>                                         |
| Let's Encrypt auto-renewal working                |  HIGH   | <input type="checkbox"/> Pending | <code>sudo certbot renew --dry-run</code>                                                          |
| CORS restricted to havenworld.pages.dev only      |  HIGH   | <input type="checkbox"/> Pending | Test with: <code>curl -H "Origin: https://evil.com" -I https://YOUR_IP.nip.io/api/health</code>    |
| Input validation (Zod) on all API endpoints       |  HIGH   | <input type="checkbox"/> Pending | Send malformed body to every endpoint; verify 400 response                                         |
| Rate limiting active on /api/auth/* (10/15min)    |  HIGH   | <input type="checkbox"/> Pending | <code>node security-tests/brute-force.js</code>                                                    |
| Rate limiting active on /api/* (100/15min)        |  MEDIUM | <input type="checkbox"/> Pending | Send 110 requests in 15 min; verify 429 after 100                                                  |
| JWT algorithm pinned to HS256                     |  HIGH | <input type="checkbox"/> Pending | Send token with <code>alg: "none"</code> ; verify rejection                                        |
| JWT_SECRET is ≥32 random bytes                    |  HIGH | <input type="checkbox"/> Pending | <code>echo -n "\$JWT_SECRET"   wc -c (≥64 chars)</code>                                            |
| Server refuses to start without JWT_SECRET        |  HIGH | <input type="checkbox"/> Pending | <code>JWT_SECRET= node dist/server.js</code> (should throw FATAL)                                  |
| Refresh token rotation working                    |  HIGH | <input type="checkbox"/> Pending | Use refresh token twice; second use should return 401                                              |
| Refresh token in httpOnly, SameSite=Strict cookie |  HIGH | <input type="checkbox"/> Pending | <code>curl -c /tmp/cookies.txt https://YOUR_IP.nip.io/api/auth/login</code> , inspect cookie flags |
| XSS prevention — chat messages sanitized          |  HIGH | <input type="checkbox"/> Pending | Send <code>&lt;script&gt;alert(1)&lt;/script&gt;</code> as chat; verify it is stripped             |
| WebSocket Origin validation (CSWSH blocked)       |  HIGH | <input type="checkbox"/> Pending | <code>wscat -c wss://YOUR_IP.nip.io --header "Origin: https://evil.com"</code>                     |

| Security Item                                                 | Risk                                                                                              | Status                           | Command to Verify                                                |
|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------|----------------------------------|------------------------------------------------------------------|
| WebSocket auth required at handshake                          |  <b>HIGH</b>     | <input type="checkbox"/> Pending | wscat -c wss://YOUR_IP.nip.io (no token — should fail)           |
| Socket.io maxHttpBufferSize set to 64KB                       |  <b>MEDIUM</b>   | <input type="checkbox"/> Pending | Send 70KB payload; verify connection closed                      |
| perMessageDeflate disabled (CRIME/BREACH)                     |  <b>MEDIUM</b>   | <input type="checkbox"/> Pending | Check Socket.io server config in code review                     |
| Ping/pong heartbeat active (pingInterval: 10s)                |  <b>LOW</b>      | <input type="checkbox"/> Pending | Check Socket.io server config in code review                     |
| Server-authoritative movement enforced                        |  <b>HIGH</b>     | <input type="checkbox"/> Pending | node security-tests/speed-hack.js                                |
| Economy atomicity — all transactions use Prisma \$transaction |  <b>HIGH</b>     | <input type="checkbox"/> Pending | node security-tests/economy-exploit.js                           |
| Admin role read from DB, never JWT                            |  <b>HIGH</b>    | <input type="checkbox"/> Pending | Forge JWT with role: "admin"; verify admin endpoint returns 403  |
| PostgreSQL localhost-only, no external access                 |  <b>HIGH</b>   | <input type="checkbox"/> Pending | sudo ss -tlnp   grep 5432 (should show 127.0.0.1 only)           |
| Redis localhost-only with requirepass                         |  <b>HIGH</b>   | <input type="checkbox"/> Pending | sudo ss -tlnp   grep 6379 (127.0.0.1 only)                       |
| Daily backup cron running and encrypted                       |  <b>HIGH</b>   | <input type="checkbox"/> Pending | sudo crontab -l   grep backup and check /var/backups/havenworld/ |
| npm audit clean (no high/critical vulnerabilities)            |  <b>HIGH</b>   | <input type="checkbox"/> Pending | npm audit --audit-level=high                                     |
| Dependabot configured in .github/dependabot.yml               |  <b>MEDIUM</b> | <input type="checkbox"/> Pending | Check GitHub repo → Insights → Dependency graph → Dependabot     |
| GPL-3.0 license compliance passing                            |  <b>MEDIUM</b> | <input type="checkbox"/> Pending | npm run check-licenses                                           |

| Security Item                                 | Risk     | Status                           | Command to Verify                                                                   |
|-----------------------------------------------|----------|----------------------------------|-------------------------------------------------------------------------------------|
| Sentry configured with PII scrubbing          | ● MEDIUM | <input type="checkbox"/> Pending | Trigger a test error; verify no tokens in Sentry event                              |
| No stack traces sent to clients in production | ● HIGH   | <input type="checkbox"/> Pending | Trigger a 500 error; verify response body has no file paths or stack                |
| Server header removed (server_tokens off)     | ● LOW    | <input type="checkbox"/> Pending | <code>curl -I https://YOUR_IP.nip.io   grep -i server</code> (should be absent)     |
| x-powered-by header removed (Helmet)          | ● LOW    | <input type="checkbox"/> Pending | <code>curl -I https://YOUR_IP.nip.io/api/health   grep -i x-powered</code> (absent) |

## Security Debt Tracking

The following items are acknowledged security improvements that are intentionally deferred to post-launch. They do not block alpha launch but must be scheduled in the first three months of live operation. Track these as GitHub Issues with the label `security-debt`.

| Item                                                | Risk Level | Rationale for Deferral                                                                                                   | Target Milestone          |
|-----------------------------------------------------|------------|--------------------------------------------------------------------------------------------------------------------------|---------------------------|
| Two-factor authentication (TOTP) for admin accounts | ● MEDIUM   | Admin team is small (1 person at alpha); SSH key + strong password is sufficient initially                               | v0.3 (30 days post-alpha) |
| Third-party penetration test                        | ● MEDIUM   | Free-tier only — professional pen testing has cost. Schedule when first revenue is received or community volunteer found | Post-public beta          |
| Bug bounty program                                  | ● LOW      | Requires legal framework and payout budget; community disclosure policy                                                  | Post-beta                 |

| Item                                                   | Risk Level | Rationale for Deferral                                                                                   | Target Milestone                |
|--------------------------------------------------------|------------|----------------------------------------------------------------------------------------------------------|---------------------------------|
|                                                        |            | sufficient at alpha                                                                                      |                                 |
| Navmesh server-side walkability validation             | ● MEDIUM   | Speed validation is implemented; navmesh import for server-side use requires separate work item (Part 9) | v0.4                            |
| IP reputation API integration (AbuseIPDB)              | ● LOW      | fail2ban covers reactive banning; proactive IP reputation requires API budget                            | Post-beta                       |
| Encrypted Redis persistence (RDB/AOF encryption)       | ● LOW      | Redis bound to localhost; no external access risk at current scale                                       | v1.0                            |
| Web Application Firewall (WAF) rules in Cloudflare     | ● MEDIUM   | Requires Cloudflare Pro (\$20/month); free tier provides basic DDoS only                                 | When revenue allows upgrade     |
| Automated quarterly backup restore test                | ● MEDIUM   | Manual restore procedure documented in Section 8.3; automation requires additional scripting effort      | v0.5                            |
| Content moderation pipeline (automated chat filtering) | ● MEDIUM   | Manual moderator review sufficient at alpha user count (<100 users)                                      | v0.4 (before open registration) |
| Security.txt file at /.well-known/security.txt         | ● LOW      | Responsible disclosure contact; low-effort but non-critical at alpha                                     | v0.3                            |

---

## What's Next — Part 9 Preview

With the security audit complete, the HavenWorld server is hardened across all five layers of the defense-in-depth model. Part 9 of this series will cover **Performance Optimization and Scalability**: Redis caching strategy for room state, PostgreSQL query optimization with `EXPLAIN ANALYZE`, Socket.io horizontal scaling with Redis

adapter, Nginx reverse proxy tuning (`worker_processes`, `keepalive`), and load testing with Artillery on the Oracle Cloud ARM instance.



#### Document Metadata

**Series:** HavenWorld Build Guide | **Part:** 8 of 10 | **Revision:** 1.0 | **Date:** September 18, 2026 | **Author:** Daniel | **Repository:** [github.com/dboone323/HavenWorld](https://github.com/dboone323/HavenWorld) | **License:** GPL-3.0 | **Status:** Pre-Alpha Security Baseline